
Eine Analyse mit dem V-Dem Datensatz zur Identifikation von Faktoren der politischen Instabilität

Adapt the headline in the cell above!

Present a short summary of the pitch! (Add up to 5 bullet points in the cell below)

- Politische Instabilität stellt ein erhebliches Risiko für Länder und die globale Sicherheit dar. Eine frühzeitige Erkennung ist wichtig
- Das Ziel des Projektes ist die Entdeckung länderspezifischer Stereotypen im Hinblick auf ihre politische Stabilität durch eine explorative Datenanalyse
- Regierungen und internationale Organisationen können dadurch gezielter präventive Maßnahmen in Risikoländern ergreifen, weil sie die Faktoren, an denen etwas geändert werden muss, kennen
- Der V-Dem-Datensatz bietet zahlreiche Daten zu politischer Instabilität, Demokratiequalität, Wirtschaft, Bildung und sozialen Indikatoren auf Intervallskalenniveau, die das Clustern und Erkennen von Ländergruppen ermöglichen

Imports and Settings

Use the following code cell to organize all imports and global settings. Do not remove the pandas option. (Commented code, in the next cell)

```
In [273... import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [274... # make pandas display ALL columns of a dataframe
pd.options.display.max_columns = None
```

Task 1 -- The Data

Present the origin of the dataset, including e.g. * the source where you got it from (link to the repository), * the author(s), * the license, * the purpose for which it was collected! (Add bullet points in the cell below)

Herkunft des Datensatzes: V-Dem Institut (Varieties of Democracy)

- **Quelle:** [V-Dem Project – Official Website](#)
- **Autoren:** Das V-Dem-Projekt wird geleitet von der V-Dem-Institution unter der Koordination von Staffan I. Lindberg (University of Gothenburg) und einem internationalen Team von Politikwissenschaftler:innen.
- **Lizenz:** Der Datensatz ist unter der [Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License \(CC BY-NC-ND 4.0\)](#) veröffentlicht. Quelle: <https://github.com/vdemoinstitute/vdemdata/releases>
- **Zweck der Datenerhebung:**
Der Datensatz wird jährlich erhoben, um detaillierte und vergleichbare Informationen über Demokratie, politische Institutionen und Menschenrechte in nahezu allen Ländern der Welt über einen langen historischen Zeitraum zu ermöglichen. Er richtet sich insbesondere an Forscher:innen, politische Analyst:innen und Institutionen zur Bewertung demokratischer Entwicklungen weltweit.

The Raw Data

Load the data and present a couple of samples. Call the dataset variable `__raw_data__`. (Commented code, add cells below)

In [275...

```
raw_data = pd.read_csv("data\V-Dem-CY-Full+Others-v15.csv")
```

```
C:\Users\Dell\AppData\Local\Temp\ipykernel_13792\3939445217.py:1: DtypeWarning: Columns (364,365,366,399,804,836,837,924,1240,1257,1486,3094,3168,3169,3341,3342,3344,3345,3347,3350,3352) have mixed types. Specify dtype option on import or set low_memory=False.
```

```
raw_data = pd.read_csv("data\V-Dem-CY-Full+Others-v15.csv")
```

In [276...

```
# Überblick über Struktur und Spalten
print(raw_data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 27913 entries, 0 to 27912
Columns: 4607 entries, country_name to e_pt_coup_attempts
dtypes: float64(4560), int64(18), object(29)
memory usage: 981.1+ MB
None
```

In [277...

```
# gibt 5 Zufällige Spalten aus
raw_data.sample(5)
```

Out[277]:

	country_name	country_text_id	country_id	year	historical_date	project	historical	his
								S
18779	Somalia	SOM	130	1906	1906-12-31	0	0	under contr
27748	Saxe-Weimar-Eisenach	SAX	365	1837	1837-12-31	1	1	Duc Saxe-V anc Ei
2597	Poland	POL	17	1863	1863-12-31	1	1	Cc [incorp ii Rt
1410	Burma/Myanmar	MMR	10	1891	1891-12-31	1	1	[Upf Lower unite
11805	Germany	DEU	77	1882	1882-12-31	1	1	C

In [278...]

```
# Ich habe direkt zum Anfang eine Auswahl von Spalten getroffen von denen ich denke
# Da der Datensatz über 4000 Spalten besitzt, war dies nötig. Für die Aufgaben wäre
# Als Kommentar markierte Variablen enthielten zu viele NaNs, weshalb ich mich nach
```

```
relevante_spalten = [
    "country_name",    # Land
    "year",            # Jahr
    "e_wbgi_pve",      # politische Stabilität
    "v2x_libdem",      # Liberale Demokratie (Index)
    "v2x_polyarchy",   # Wahlen (Index)
    "v2x_freexp_altinf", # Meinungs- und Informationsfreiheit (Index)
    "v2xcl_ro1",       # Rechtsstaatlichkeit (Index)
    "v2xcl_acjst",     # Zugang zur Justiz
    "v2x_rule",        # Rule of Law (Index)
    "v2x_pubcorr",     # Öffentliche Korruption
    "v2x_corr",        # politische Korruption
    "v2x_execorr",     # Korruption der Exekutive
    # "e_cow_exports",  # Exporte
    # "e_cow_imports",  # Importe
    # "e_gdp",          # BIP
    # "e_miinflat",     # Inflation
    "e_miferrat",      # Fertilitätsrate
    "e_pelifeex",      # Lebenserwartung
    # "e_civil_war",    # Bürgerkrieg
    "v2cacamps_mean",  # politische Polarisierung
    "v2caviol_mean",   # politische Gewalt
    "v2caassemb_mean", # Versammlungsfreiheit
    "v2cagenmob_mean", # Massenmobilisierung
    "v2caautmob_mean", # Autokratische Bewegungen
    "v2cademmob_mean", # Demokratische Bewegungen
    "v2castate_mean",  # Beteiligung an staatlich organisierten Gruppen
    "v2canonpol_mean", # Beteiligung an unabhängigen nicht-politischen Gruppen
    "v2cafres_mean"    # Freiheit von Forschung und Lehre
]

relevante_spalten = raw_data[relevante_spalten]
```

In [279...]

```
# Daten von 2019 bis 2024 filtern
letzten_fünf_jahre = relevante_spalten[
    relevante_spalten["year"].between(2019, 2024)
]

letzten_fünf_jahre.sample(5)
```

Out[279]:

	country_name	year	e_wbgi_pve	v2x_libdem	v2x_polyarchy	v2x_freexp_altinf	v2xcl_ro1
8585	Indonesia	2022	-0.439	0.393	0.557	0.771	0.736
2533	Colombia	2024	NaN	0.557	0.701	0.843	0.700
4567	Pakistan	2022	-1.900	0.234	0.383	0.638	0.291
14357	Papua New Guinea	2023	NaN	0.377	0.467	0.808	0.687
14949	Syria	2024	NaN	0.054	0.145	0.069	0.097

Explain the dataset with all its features. * The features should be described in the following table. The range is the set of possible values that a feature can take, e.g. "red, green, blue", " $\mathbb{R}$ ", " $1,2,\dots,12$ ". * You may describe each feature individually or groups of features, if many features are similar (whatever is easier to explain.) (Fill in the table, add bullet points in the next cell)

- Viele Variablen wurden ursprünglich z.B. durch Expertenmeinungen ordinalskaliert erhoben. Anschließend wurden diese Daten in Intervallskalen überführt (Oft auch durch die Bildung von Mittelwerten). Der Datensatz enthält daher Zusätze wie `_ord` oder `_mean`, die verschiedene Skalenniveaus abdecken. Für meine Zwecke werde ich immer Daten nutzen, die auf Intervallskalenniveau vorhanden sind. Neben Daten die auf dem Intervallskalenniveau (meist 0-4) erhoben wurden, gibt es auch indexvariablen, die sich aus verschiedenen Variablen zusammensetzen. Diese haben eine Range von 0-1.

Feature	Wertebereich	Erklärung
country_name	Länder (Strings)	Name des Landes
year	1789 – 2024	Jahr der Messung
e_wbgi_pve	$\mathbb{R}$ (ca. -4 bis 2)	Indikator für politische Stabilität (World Bank Governance Indicator)
v2x_libdem	[0, 1]	Index für liberale Demokratie
v2x_polyarchy	[0, 1]	Index für wahlbasierte Demokratie (Polyarchie)
v2x_freexp_altinf	[0, 1]	Grad der Meinungs- und Informationsfreiheit
v2xcl_rol	[0, 1]	Rechtsstaatlichkeit (Rule of Law)
e_cow_exports	$\mathbb{R}^+$ (%)	Exportquote am BIP
e_cow_imports	$\mathbb{R}^+$ (%)	Importquote am BIP
e_gdp	$\mathbb{R}^+$	Bruttoinlandsprodukt pro Kopf (meist in USD, ggf. logarithmiert)
e_miinflat	$\mathbb{R}$	Jährliche Inflationsrate
e_miferrat	$\mathbb{R}^+$	Fertilitätsrate (Kinder pro Frau)
e_pelifeex	$\mathbb{R}^+$	Lebenserwartung bei Geburt (in Jahren)
e_civil_war	{0, 1}	Bürgerkrieg im Jahr vorhanden (1 = ja)
v2cacamps_mean	[0, 4]	Politische Polarisierung
v2caviol_mean	[0, 4]	Politische Gewalt (Experteneinschätzung)
v2caassemb_mean	[0, 4]	Versammlungsfreiheit (Einschätzung)
v2cagenmob_mean	[0, 4]	Mobilisierung durch Protestbewegungen
v2caautmob_mean	[0, 4]	Autokratische soziale Bewegungen
v2cademmob_mean	[0, 4]	Demokratische soziale Bewegungen
v2castate_mean	[0, 4]	Fähigkeit des Staates, Autorität auszuüben
v2canonpol_mean	[0, 4]	Aktivität nichtpolitischer Organisationen
v2cafres_mean	[0, 4]	Ausmaß staatlicher Repression

Explain why the dataset is suitable for the pitched task! (Add up 3 bullet points in the cell below)

- Der V-Dem-Datensatz bietet zahlreiche Daten zu politischer Instabilität, Demokratiequalität, Wirtschaft, Bildung und sozialen Indikatoren auf Intervallskalenniveau, die das Clustern und Erkennen von Ländergruppen ermöglichen.
- Eine Erstellung von Stereotypen durch verschiedene Clusteranalysen bezüglich der Variable der politischen Stabilität (und anderen) ist möglich.
 - Es könnten bezüglich der Variablen Gemeinsamkeiten und Zusammenhänge gefunden werden

Task 2 -- Initial Data Analysis

Initial Data Analysis (IDA): Present relevant quantities about the data. (Commented Code + bullet points, add cells below)

```
In [280... # Anzahl der Zeilen und Spalten
raw_data.shape
```

```
Out[280]: (27913, 4607)
```

```
In [281... # Alle Länder zählen
# ISO 3166-1 führt 249 Ländercodes, inklusive abhängiger Gebiete und Sonderverwaltu
# Allgemein sind aber nur 195 Länder anerkannt.
raw_data["country_name"].nunique()
```

```
Out[281]: 202
```

```
In [282... # Statistische Kennzahlen für numerische Spalten
relevante_spalten.describe()
```

```
Out[282]:
```

	year	e_wbgi_pve	v2x_libdem	v2x_polyarchy	v2x_freexp_altinf	v2xcl_rol	
count	27913.000000	4123.000000	25530.000000	26775.000000	27377.000000	27795.000000	27
mean	1929.594096	-0.172125	0.221204	0.262252	0.419814	0.452601	
std	64.943165	0.970291	0.231332	0.262190	0.310917	0.304654	
min	1789.000000	-3.313000	0.005000	0.006000	0.010000	0.000000	
25%	1881.000000	-0.771000	0.057000	0.055000	0.132000	0.173000	
50%	1940.000000	-0.094000	0.124000	0.167000	0.350000	0.412000	
75%	1985.000000	0.594000	0.296000	0.371000	0.710000	0.721000	
max	2024.000000	1.759000	0.897000	0.923000	0.989000	0.993000	

fehlende Werte

- Die Analyse von fehlenden Werten hinsichtlich relevanter Variablen ist besonders wichtig für mein Projekt. Die Länder lassen sich nur vernünftig Clustern, wenn es genug Daten gibt. Ich muss wissen, welche Häufigkeiten bezüglich einer Variable je Land existieren.
- Ein Weiterer Punkt ist, dass die Erhebung der Variablen unterschiedliche Start und Endzeitpunkte haben kann, sodass sich die Datenabdeckung je nach Zeitraum erheblich verändern kann.
- Ich möchte primär Wissen wie sich die Datenabdeckung der Zielvariable e_wbgi_pve (politische Stabilität) verhält und in welchem Zeitraum und in Welchen Ländern die erklärenden Variablen eine Clusteranalyse zulassen.

```
In [283... # Überblick über fehlende Werte jeder Spalte  
fehlende_werte = relevante_spalten.isnull().sum()  
fehlende_werte
```

```
Out[283]: country_name          0  
year              0  
e_wbgi_pve        23790  
v2x_libdem        2383  
v2x_polyarchy     1138  
v2x_freexp_altinf  536  
v2xcl_rol         118  
v2xcl_acjst       209  
v2x_rule          388  
v2x_pubcorr       544  
v2x_corr          893  
v2x_execorr       696  
e_miferrat        2596  
e_pelifeex        2596  
v2cacamps_mean    8562  
v2caviol_mean     8562  
v2caassemb_mean   8562  
v2cagenmob_mean   8562  
v2caautmob_mean   8562  
v2cademmob_mean   8562  
v2castate_mean    8562  
v2canonpol_mean   8562  
v2cafres_mean     12579  
dtype: int64
```

```
In [284... # Datenabdeckung checken  
# Für jede Spalte zählen, wie viele Nicht-Null-Werte pro Land vorhanden sind  
datenabdeckung = relevante_spalten.groupby("country_name").count()  
datenabdeckung
```


Out[284]:

	year	e_wbgi_pve	v2x_libdem	v2x_polyarchy	v2x_freexp_altinf	v2xcl_rol	v2xcl_acj
country_name							
Afghanistan	236	24	236	236	236	236	236
Albania	113	24	111	111	113	113	113
Algeria	125	24	125	125	125	125	125
Angola	125	24	125	125	125	125	125
Argentina	236	24	236	236	236	236	236
...	...	...	...	...	...	...	...
Württemberg	83	0	57	57	83	83	83
Yemen	169	24	169	169	169	169	169
Zambia	114	24	114	114	114	114	114
Zanzibar	169	0	125	125	169	169	169
Zimbabwe	125	24	125	125	125	125	125

202 rows × 22 columns

In [285...]

```
# Datenabdeckung der letzten Jahre
filtered = relevante_spalten[relevante_spalten["year"].between(1996, 2022)]

# Danach Gruppierung und Zählung der Nicht-Null-Werte je Land und Spalte
datenabdeckung1 = filtered.groupby("country_name").count()

# Ergebnis anzeigen
datenabdeckung1.head()
```

Out[285]:

	year	e_wbgi_pve	v2x_libdem	v2x_polyarchy	v2x_freexp_altinf	v2xcl_rol	v2xcl_acj
country_name							
Afghanistan	27	24	27	27	27	27	27
Albania	27	24	27	27	27	27	27
Algeria	27	24	27	27	27	27	27
Angola	27	24	27	27	27	27	27
Argentina	27	24	27	27	27	27	27

Wichtige Erkenntnisse:

- Es gibt keine Informationen bezüglich der politischen Stabilität zu Palestine/Gaza, zu Palestine/ Westbank, zu Hong Kong, zum Kosovo, zum Somaliland und zu Sansibar
- Die allermeisten Länder haben einen Erhebungszeitraum von 1996 bis 2022 --> Es werden nicht mehr Daten wenn man den Zeitraum erweitert. Obwohl der Zeitraum 27 Jahre umfasst, haben die meisten Länder 24 einträge, weshalb die Erhebung scheinbar 3 mal ausgesetzt worden ist. Die Daten zu einigen wenigen Ländern wurden zudem seltener als 24 mal erhoben. Der SüdSudan wurden nur 12 mal erhoben. Timor-Leste 21 mal, Serbia und Montenegro 17 mal.
- In vielen Anderen Variablen erscheint die Datendichte gut zu sein. Jedoch nicht in allen Variablen. So weist die Variable zum Civilwar, die Variable zum Import, die Variable zum Export und die Variable zur Inflation auf ein paar defizite hin. # --> habe ich nachträglich entfernt. Daher tauchen diese hier gar nicht mehr auf.

Task 3 -- Preprocessing

Conduct all steps of Task 3. Call the result "data". (Commented code + bullet points, add cells below)

- Ich möchte direkt wissen wofür die einzelnen Spaltennamen stehen. --> Umbenennen
- Ich möchte einen Datensatz mit einer möglichst hohen Datendichte, der die letzten Jahrzehnte abbildet.
- Das Problem ist, dass einige wenige Länder weniger Daten enthalten als andere. Die Spalten sollten jedoch für jedes Land gleichviele Daten bereitstellen.
- Natürlich könnte man nur Daten aus einem Jahr nehmen und hätte dann einen ausgewogenen Datensatz. Jedoch erwarte ich eine bessere und fundiertere Analyse, wenn auch die Vergangenheit analysiert wird.
- Da kein Land im Jahr 1997, 1999, 2001, 2023 und 2024 angaben bezüglich der Hauptvariable (politische Stabilität) besitzt, möchte ich diese Zeilen löschen.
- Ferner sollen auch Länderzeilen denen kein Wert bezüglich der Spalte der politischen Stabilität zugeordnet ist gelöscht werden.
- Bei Ländern mit weniger Werten habe ich lange überlegt, ob man eine Spline-Interpolation verwenden könnte. Eine Spline-Interpolation würde dafür sorgen, dass möglichst realistische Verläufe zwischen den Zeitreihenwerten erzeugt werden. Wenn aber mehrere aufeinander folgende Werte fehlen, dann funktioniert dies nicht. Weil das immer der Fall war, habe ich mich dazu entschieden diese Länder ebenso rauszulöschen.
- Bezüglich der anderen Variablen kann es ebenso vorkommen, dass es nicht gleichviele Zeilen je Land gibt bzw.. das einige Länder gar nicht vertreten sind, weshalb einige wenige Länder schwächer ins Gewicht fallen könnten. Da dies bei einigen Variablen sehr extrem war, habe ich mich entschieden diese Variablen zu löschen bzw.. gar nicht erst zu laden.

In [286...

Umbenennen der Spaltennamen

```
Test = filtered.rename(columns={
    "country_name": "Land",
    "year": "Jahr",
    "e_wbgi_pve": "politische Stabilität",
    "v2x_libdem": "Liberales Demokratie (Index)",
    "v2x_polyarchy": "Wahlen (Index)",
    "v2x_freexp_altinf": "Meinungs- und Informationsfreiheit (Index)",
    "v2xcl_ro1": "Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)",
    "v2xcl_acjst": "Zugang zur Justiz",
    "v2x_rule": "Rechtsstaatlichkeit(Index)",
    "v2x_pubcorr": "Öffentliche Korruption",
    "v2x_corr": "politische Korruption",
    "v2x_execorr": "Korruption der Exekutive",
    "e_cow_exports": "Exporte",
    "e_cow_imports": "Importe",
    "e_gdp": "BIP",
    "e_miinflat": "Inflation",
    "e_miferrat": "Fertilitätsrate",
    "e_pelifeex": "Lebenserwartung",
    "e_civil_war": "Bürgerkrieg",
    "v2cacamps_mean": "politische Polarisierung",
    "v2caviol_mean": "politische Gewalt",
    "v2caassemb_mean": "Versammlungsfreiheit",
    "v2cagenmob_mean": "Massenmobilisierung",
    "v2caautmob_mean": "Autokratische Bewegungen",
    "v2cademmob_mean": "Demokratische Bewegungen",
    "v2castate_mean": "Beteiligung an staatlich organisierten Gruppen",
    "v2canonpol_mean": "Beteiligung an unabhängigen nicht-politischen Gruppen",
    "v2cafres_mean": "Freiheit von Forschung und Lehre"
})
```

In [287...

Löschen der besagten Jahreszeilen

```
Test_cleaned = Test[~Test["Jahr"].isin([1997, 1999, 2001, 2023, 2024])]
```

In [288...

Länderzeilen aufgrund mangelhafter Daten löschen.

erstellen einer Liste dieser Länder

```
raus = ["Palestine/Gaza", "Palestine/West Bank", "Hong Kong", "Kosovo", "Somaliland",
```

Entfernen der Zeilen aus dem DF. ~ ist das Zeichen für Negation. Es bleiben also

```
Test = Test_cleaned[~Test_cleaned["Land"].isin(raus)]
```

In [289...

```
Data = Test.dropna()
```

In [290...

```
Data = Test
```

- Vorstellen der relevanten Kenngrößen, des veränderten finalen DFs.

In [291...

Der DF besitzt 4044 Zeilen und 28 Spalten

```
Data.shape
```

Out[291]: (4056, 23)

In [292...

```
# Insgesamt 26342 Einträge
Data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 4056 entries, 207 to 26342
Data columns (total 23 columns):
#   Column                                                                 Non-Null Count
Dtype
---  -----
0   Land                                                                    4056 non-null
object
1   Jahr                                                                    4056 non-null
int64
2   politische Stabilität                                                  4056 non-null
float64
3   Liberale Demokratie (Index)                                           4053 non-null
float64
4   Wahlen (Index)                                                        4056 non-null
float64
5   Meinungs- und Informationsfreiheit (Index)                            4056 non-null
float64
6   Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)        4056 non-null
float64
7   Zugang zur Justiz                                                      4056 non-null
float64
8   Rechtsstaatlichkeit(Index)                                             4056 non-null
float64
9   öffentliche Korruption                                                4056 non-null
float64
10  politische Korruption                                                  4050 non-null
float64
11  Korruption der Exekutive                                               4056 non-null
float64
12  Fertilitätsrate                                                        4056 non-null
float64
13  Lebenserwartung                                                        4056 non-null
float64
14  politische Polarisierung                                               4056 non-null
float64
15  politische Gewalt                                                       4056 non-null
float64
16  Versammlungsfreiheit                                                  4056 non-null
float64
17  Massenmobilisierung                                                    4056 non-null
float64
18  Autokratische Bewegungen                                               4056 non-null
float64
19  Demokratische Bewegungen                                               4056 non-null
float64
20  Beteiligung an staatlich organisierten Gruppen                       4056 non-null
float64
21  Beteiligung an unabhängigen nicht-politischen Gruppen                 4056 non-null
float64
22  Freiheit von Forschung und Lehre                                       4019 non-null
float64
dtypes: float64(21), int64(1), object(1)
memory usage: 760.5+ KB
```

In [293...

```
# Übersicht über deskriptive Statistiken
Data.describe()
```

Out[293]:

	Jahr	politische Stabilität	Liberales Demokratie (Index)	Wahlen (Index)	Meinungs- und Informationsfreiheit (Index)	Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	Zug
count	4056.000000	4056.000000	4053.000000	4056.000000	4056.000000	4056.000000	4056
mean	2010.250000	-0.167449	0.406576	0.520684	0.666696	0.672943	0
std	7.339445	0.970768	0.271479	0.264959	0.287130	0.269890	0
min	1996.000000	-3.313000	0.005000	0.015000	0.012000	0.006000	0
25%	2004.750000	-0.772000	0.150000	0.283000	0.504750	0.492000	0
50%	2010.500000	-0.092500	0.371000	0.525500	0.759000	0.741000	0
75%	2016.250000	0.605000	0.653000	0.772000	0.896000	0.909000	0
max	2022.000000	1.759000	0.897000	0.923000	0.989000	0.993000	0

- Datendichte

In [294...

```
# Entscheidend ist wie viele Länder die gleiche Anzahl an Einträgen je Spalte haben
# Der Code soll folgendes Aussagen: z.B. in Der Spalte Liberale Demokratie (Index)

for spalte in Data.columns: # Schleife die durch alle Columns des Dfs Data iteriert
    print(Data.groupby("Land")[spalte].count().value_counts(), "\n")

    # Zuerst wird nach Land gruppiert. Dann wird auf jede Spalte zugegriffen und
    # value_counts zählt dann, wie oft jede mögliche Anzahl von Werten für die
    # 0 müsste bedeuten, z.B. bei Exporte, dass es zu drei Ländern keine Daten
```

Land
24 169
Name: count, dtype: int64

Jahr
24 169
Name: count, dtype: int64

politische Stabilität
24 169
Name: count, dtype: int64

Liberale Demokratie (Index)
24 168
21 1
Name: count, dtype: int64

Wahlen (Index)
24 169
Name: count, dtype: int64

Meinungs- und Informationsfreiheit (Index)
24 169
Name: count, dtype: int64

Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)
24 169
Name: count, dtype: int64

Zugang zur Justiz
24 169
Name: count, dtype: int64

Rechtsstaatlichkeit(Index)
24 169
Name: count, dtype: int64

öffentliche Korruption
24 169
Name: count, dtype: int64

politische Korruption
24 168
18 1
Name: count, dtype: int64

Korruption der Exekutive
24 169
Name: count, dtype: int64

Fertilitätsrate
24 169
Name: count, dtype: int64

Lebenserwartung
24 169
Name: count, dtype: int64

politische Polarisierung

```
24    169
Name: count, dtype: int64
```

politische Gewalt

```
24    169
Name: count, dtype: int64
```

Versammlungsfreiheit

```
24    169
Name: count, dtype: int64
```

Massenmobilisierung

```
24    169
Name: count, dtype: int64
```

Autokratische Bewegungen

```
24    169
Name: count, dtype: int64
```

Demokratische Bewegungen

```
24    169
Name: count, dtype: int64
```

Beteiligung an staatlich organisierten Gruppen

```
24    169
Name: count, dtype: int64
```

Beteiligung an unabhängigen nicht-politischen Gruppen

```
24    169
Name: count, dtype: int64
```

Freiheit von Forschung und Lehre

```
24    164
20      2
17      1
12      1
14      1
Name: count, dtype: int64
```

In [295...

```
# Überblick über fehlende Werte! --> Export, Import, inflation, BIP und Bürgerkrieg
fehlende_werte1 = Data.isnull().sum()
fehlende_werte1
```



```
Out[295]: Land 0
Jahr 0
politische Stabilität 0
Liberale Demokratie (Index) 3
Wahlen (Index) 0
Meinungs- und Informationsfreiheit (Index) 0
Gleichheit vor dem Gesetz und individuelle Freiheiten (Index) 0
Zugang zur Justiz 0
Rechtsstaatlichkeit(Index) 0
öffentliche Korruption 0
politische Korruption 6
Korruption der Exekutive 0
Fertilitätsrate 0
Lebenserwartung 0
politische Polarisierung 0
politische Gewalt 0
Versammlungsfreiheit 0
Massenmobilisierung 0
Autokratische Bewegungen 0
Demokratische Bewegungen 0
Beteiligung an staatlich organisierten Gruppen 0
Beteiligung an unabhängigen nicht-politischen Gruppen 0
Freiheit von Forschung und Lehre 37
dtype: int64
```

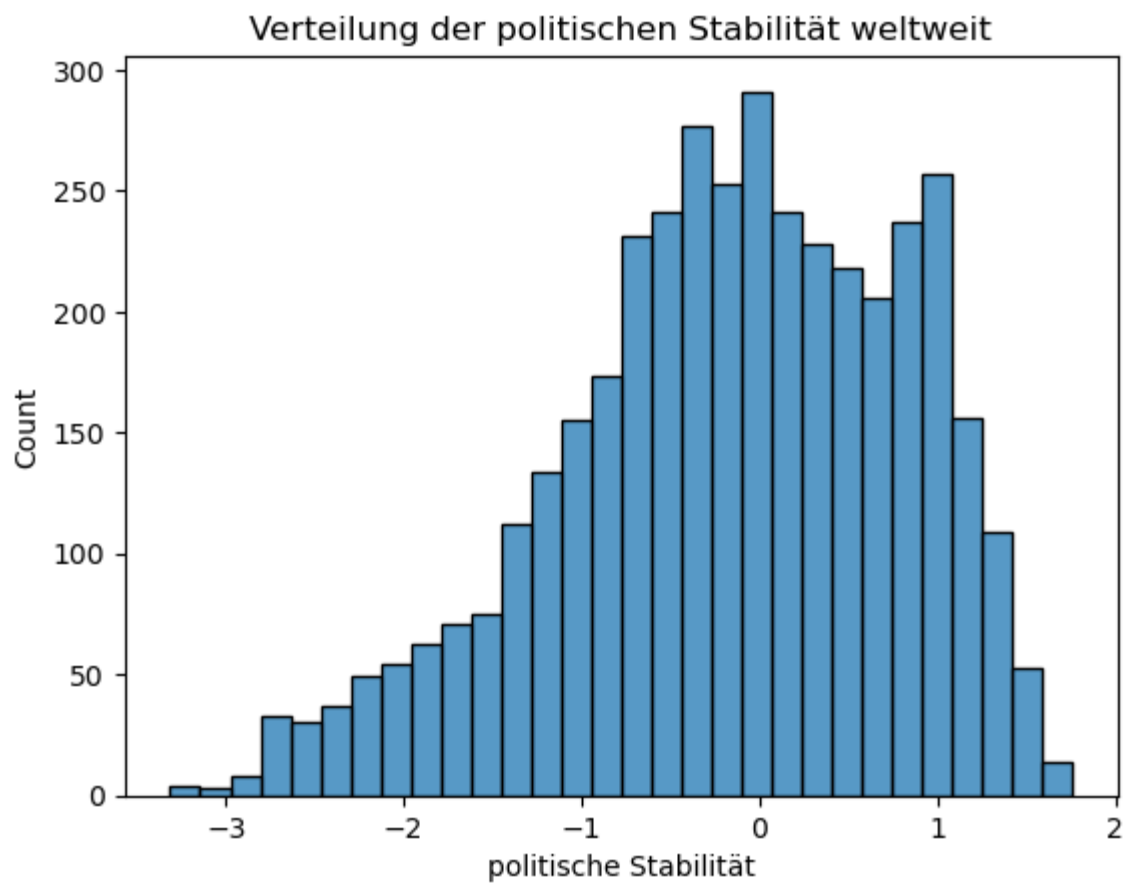
```
In [296... Data = Data.dropna()
# Es ist vertretbar die paar Zeilen zu löschen.
```

Task 4 -- Exploratory Data Analysis

Erkunden der Zielvariable

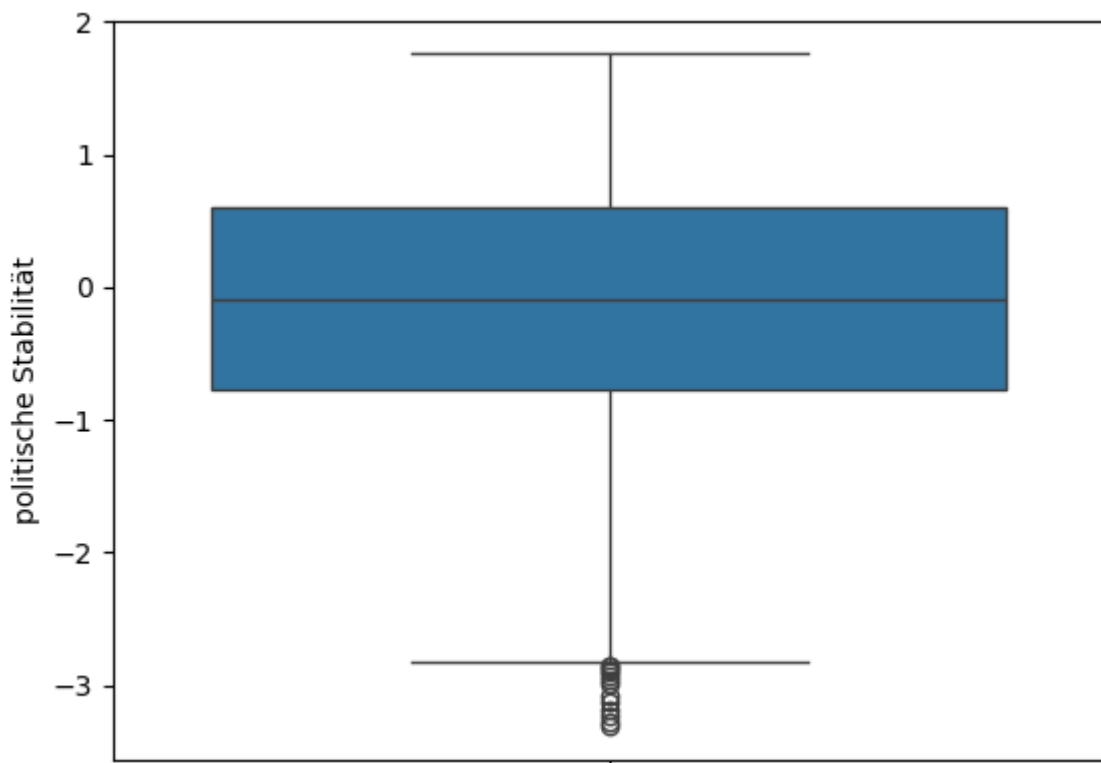
- Als erstes möchte ich die Zielvariable genauer beleuchten. Dazu möchte ich eine Verteilung erstellen.
- Der Sinn dahinter ist, dass ich dadurch sehen könnte wo sich Werte zentrieren/ wie sich die Werte verteilen.
- Danach möchte ich einen Boxplot erstellen um Ausreißer zu entdecken und um zu wissen von wo bis wo der 50% Bereich geht.
- In einem zweiten Schritt will ich mir dann angucken ob es eine allgemeinen Trend über die Jahre gibt.

```
In [297... # Histogramme zur Verteilung
sns.histplot(Data["politische Stabilität"])
plt.title("Verteilung der politischen Stabilität weltweit")
plt.show()
```



In [298...

```
# Boxplot um Ausreißer zu entdecken  
sns.boxplot(Data["politische Stabilität"])  
plt.show()
```



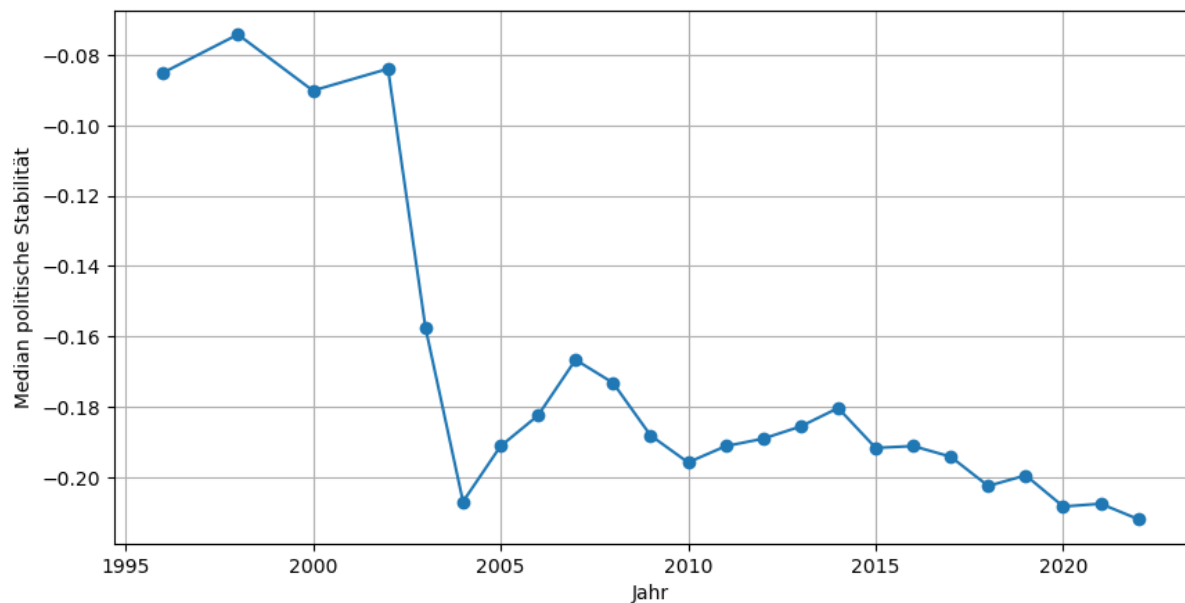
```
In [299... Data['politische Stabilität'].median()
```

```
Out[299]: -0.098
```

- Aus der Verteilung und dem Boxplot lässt sich entnehmen, dass sich 50 % der Daten (also der Interquartilsabstand, IQR) etwa zwischen 0.75 und 0.85 befinden. Der Median, der als Linie innerhalb der Box dargestellt ist, liegt bei etwa -0.0925 und zeigt damit, dass mehr als die Hälfte der Werte unter null, also im negativen Bereich liegen. Ausreißer, also Werte außerhalb von $1.5 \times \text{IQR}$, befinden sich ausschließlich im negativen Bereich. Das bedeutet, es gibt einige Länder und Jahre, die besonders instabil sind. Auffällig ist, dass die Streuung im negativen Bereich deutlich größer ist als im positiven. Es gibt vereinzelte Fälle extremer Instabilität, aber kein einziges Land, das ebenso stark ins Positive abweicht – die stabilsten Länder sind weniger extrem stabil als die instabilsten instabil sind.

```
In [300... # Durchschnittliche politische Stabilität pro Jahr berechnen und Plotten
stabilität_zeit = Data.groupby('Jahr')['politische Stabilität'].mean()
# gruppieren nach Jahr und für jedes Jahr in der Spalte politische Stabilität d

# plotten
plt.figure(figsize=(10, 5))
plt.plot(stabilität_zeit, marker = "o")
plt.xlabel('Jahr')
plt.ylabel('Median politische Stabilität')
plt.grid(True) # kannte ich vorher nicht. Erstellt das Raster im Hintergrund.
plt.show()
```



- man sieht, dass die politische Stabilität nie wieder auf das Niveau von vor dem 11. September 2001 zurückklettern konnte. Von 1996 bis 2022 viel das durchschnittliche Niveau mit einem sehr starken Fall von 2001 bis 2004 und trotz einer kurzen Erholung von 2005 bis 2007 von -0.06 auf - 0.21
- Krieg und Terror haben daher einen großen Einfluss auf die politische Stabilität der Welt.

Korrelationen zwischen Faktoren und politischer Stabilität

- Ich möchte wissen welche Variablen den stärksten Einfluss bzw. Zusammenhang mit der politischen Stabilität haben. Dazu möchte ich alle Variablen mit der Zielvariable korrelieren lassen. Dadurch kann man später gezielter nach Clustern suchen und weiß welche Variablen relevant sein werden.

In [301...

```
# Liste aller relevanten Variablen --> alle außer "Land" und "Jahr" und politische
# politische Stabilität muss enthalten sein, weil man diese Variable zur Berechnung
relevante_variablen = [
    'Liberales Demokratie (Index)', 'Wahlen (Index)', 'Meinungs- und Informationsfrei-
    'Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)', 'Zugang zur Jus-
    'öffentliche Korruption', 'politische Korruption', 'Korruption der Exekutive',

    # 'Exporte', 'Importe', 'BIP', 'Inflation', 'Bürgerkrieg',
    'Fertilitätsrate', 'Lebenserwartung', 'politische Polarisierung', 'politische G-
    'Versammlungsfreiheit', 'Massenmobilisierung', 'Autokratische Bewegungen',
    'Demokratische Bewegungen', 'Beteiligung an staatlich organisierten Gruppen',
    'Beteiligung an unabhängigen nicht-politischen Gruppen', 'Freiheit von Forschun-
    "politische Stabilität"
]
```

In [302...

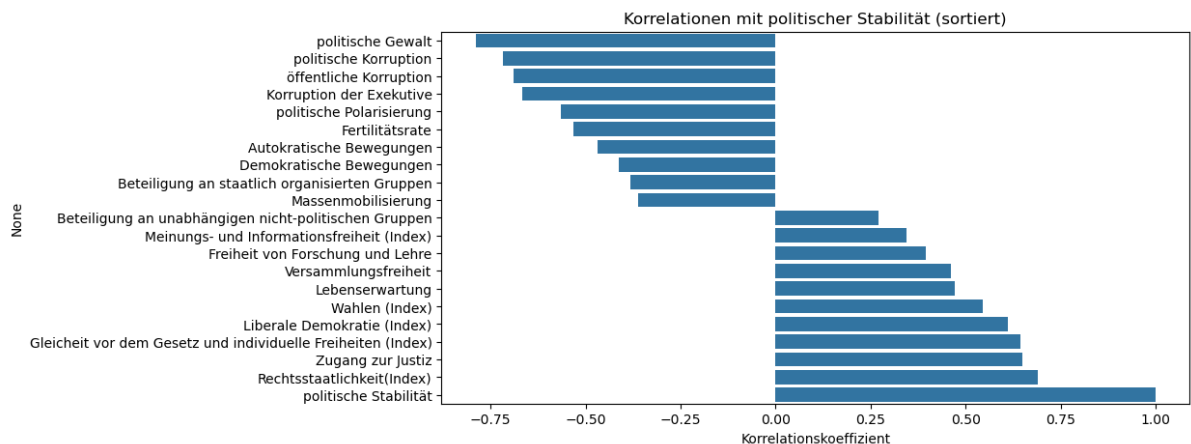
```
# Korrelation jeder Variable mit "politische Stabilität"
Korrelationen = Data[relevante_variablen].corr()['politische Stabilität']
```

In [303...

```
# sortieren
Korrelationen_sorted = Korrelationen.sort_values()
Korrelationen_sorted
```

```
Out[303]: politische Gewalt -0.789834
politische Korruption -0.718109
öffentliche Korruption -0.689218
Korruption der Exekutive -0.667066
politische Polarisierung -0.564798
Fertilitätsrate -0.532793
Autokratische Bewegungen -0.468618
Demokratische Bewegungen -0.411852
Beteiligung an staatlich organisierten Gruppen -0.382840
Massenmobilisierung -0.362440
Beteiligung an unabhängigen nicht-politischen Gruppen 0.271101
Meinungs- und Informationsfreiheit (Index) 0.343190
Freiheit von Forschung und Lehre 0.395799
Versammlungsfreiheit 0.460383
Lebenserwartung 0.471214
Wahlen (Index) 0.545407
Libérale Demokratie (Index) 0.612404
Gleichheit vor dem Gesetz und individuelle Freiheiten (Index) 0.645330
Zugang zur Justiz 0.649881
Rechtsstaatlichkeit(Index) 0.690315
politische Stabilität 1.000000
Name: politische Stabilität, dtype: float64
```

```
In [304... plt.figure(figsize=(10, 5))
sns.barplot(x=Korrelationen_sorted.values, y=Korrelationen_sorted.index)
# y=Korrelationen_sorted.index --> Das sind die Variablennamen, x= Korrelationen_so
#Werte(Korrelationskoeffizienten)
plt.xlabel('Korrelationskoeffizient')
plt.title('Korrelationen mit politischer Stabilität (sortiert)')
plt.show()
```



- Man kann erkennen, dass die Variable zur politischen Gewalt und die Variablen zur Korruption einen extrem großen Einfluss haben. Diese Variablen haben eine negative Korrelation. Es kann geschlussfolgert werden: Je stärker die politische Gewalt und je höher die Korruptionen, desto niedriger die politische Stabilität. Diese Variablen scheinen besonders wichtig und weisen auf einen negativen Zusammenhang hin
- Auf einen sehr starken positiven Zusammenhang, weisen die Variablen liberale Demokratie, die Gleichheit vor dem Gesetz und individuelle Freiheiten, Zugang zur Justiz und die Rechtsstaatlichkeit hin. --> Wenn diese Variablen einen positiven Wert haben, haben die dazugehörigen Werte der Variable der politischen Stabilität ebenso positive Werte.
- Erstere erklären instabile Länder, letztere erklären politisch stabile Länder.
- Erstaunlich ist, dass die Wirtschaftsoperatoren scheinbar einen geringen Einfluss haben. --> habe ich aber nachträglich rausgeworfen
- (P.S.: Ich habe alles nochmal etwas genauer im EDA Highlight 1 beschrieben, Final stage)

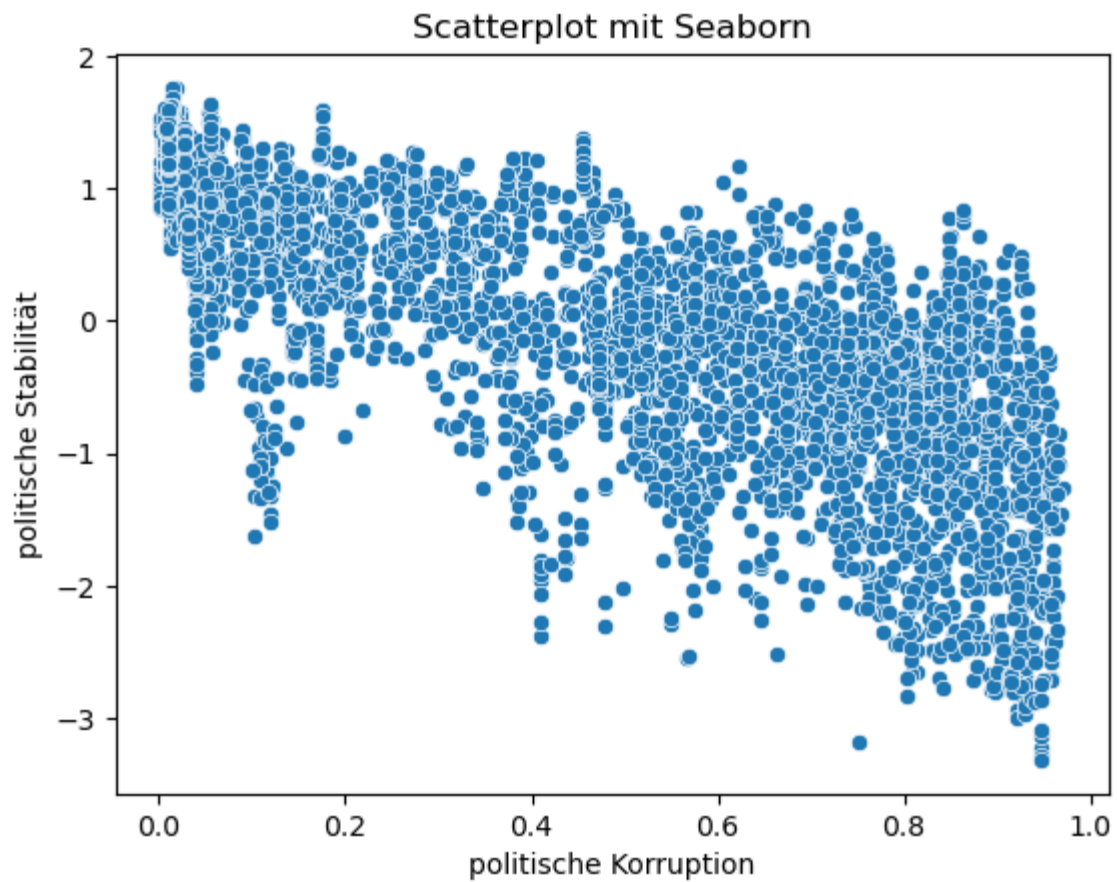
Punktewolken

In [305...

```
sns.scatterplot(data=Data, x="politische Korruption", y='politische Stabilität')

# Titel und Achsenbeschriftung
plt.title('Scatterplot mit Seaborn')
plt.xlabel('politische Korruption')
plt.ylabel('politische Stabilität')

# Zeige das Diagramm
plt.show()
```

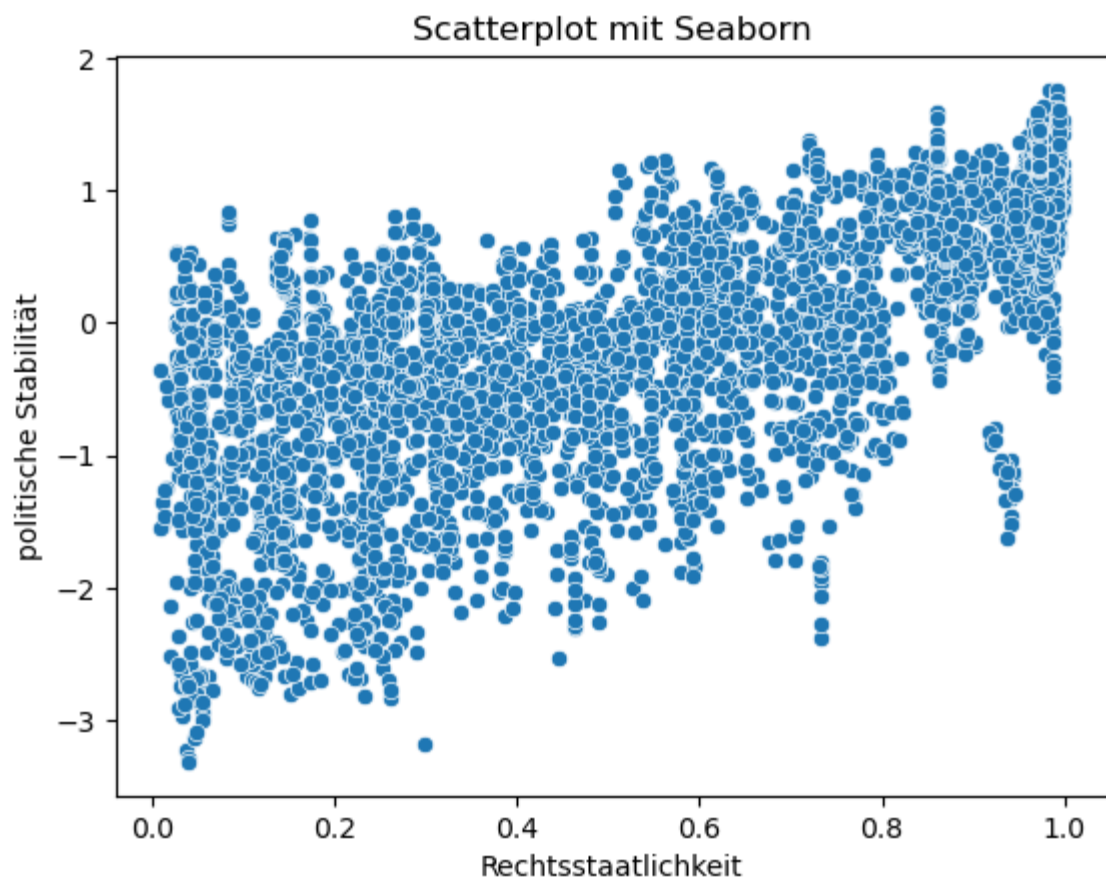


In [306...

```
sns.scatterplot(data=Data, x="Rechtsstaatlichkeit(Index)", y='politische Stabilität')

# Titel und Achsenbeschriftung
plt.title('Scatterplot mit Seaborn')
plt.xlabel('Rechtsstaatlichkeit')
plt.ylabel('politische Stabilität')

# Zeige das Diagramm
plt.show()
```

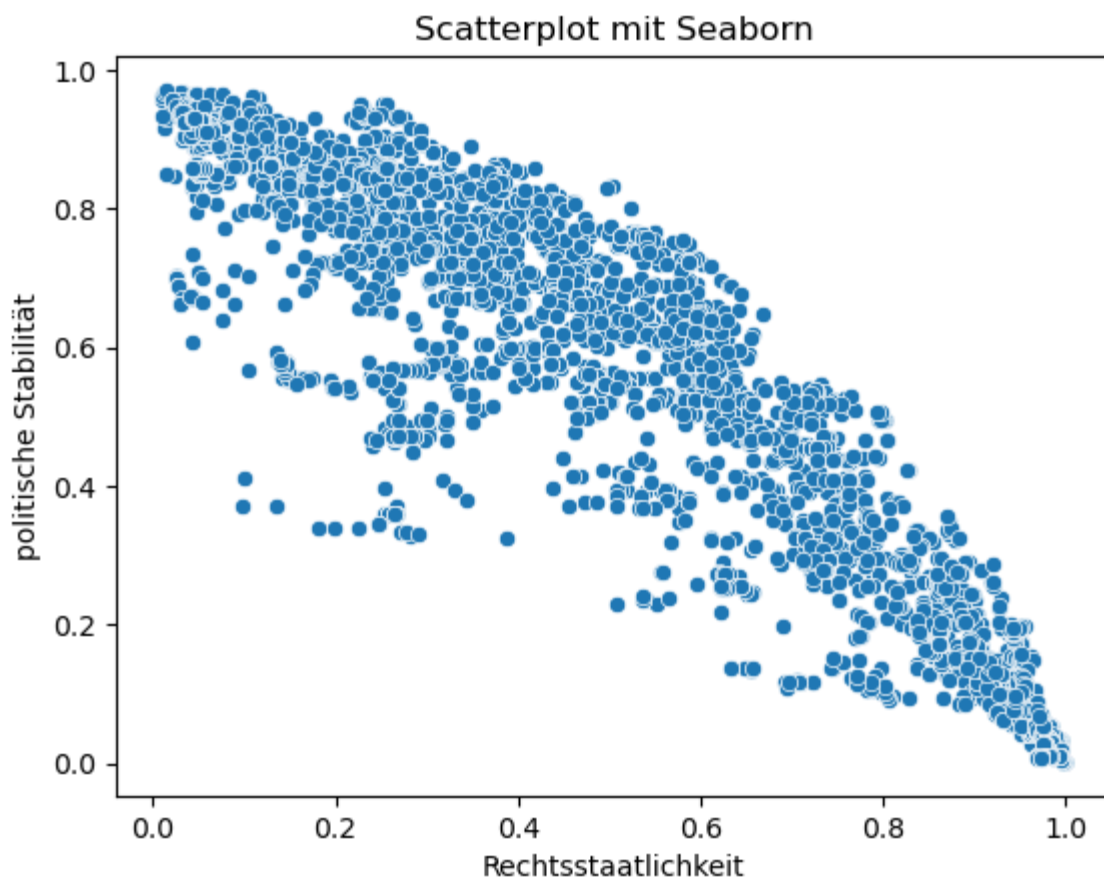


In [307...

```
sns.scatterplot(data=Data, x="Rechtsstaatlichkeit(Index)", y='politische Korruption')

# Titel und Achsenbeschriftung
plt.title('Scatterplot mit Seaborn')
plt.xlabel('Rechtsstaatlichkeit')
plt.ylabel('politische Stabilität')

# Zeige das Diagramm
plt.show()
```

Der letzte Plot zeigt, wie sich die Daten verhalten könnten, wenn alle Features in eine Clusteranalyse mit eingebunden sind. Die Daten weisen gemäß der Korrelation zur politischen Stabilität auf ein zweidimensionales Spektrum zwischen liberal, rechtsstatlich und autoritär korrupt hin.

Gruppieren nach stark negativen Korrelationsvariablen

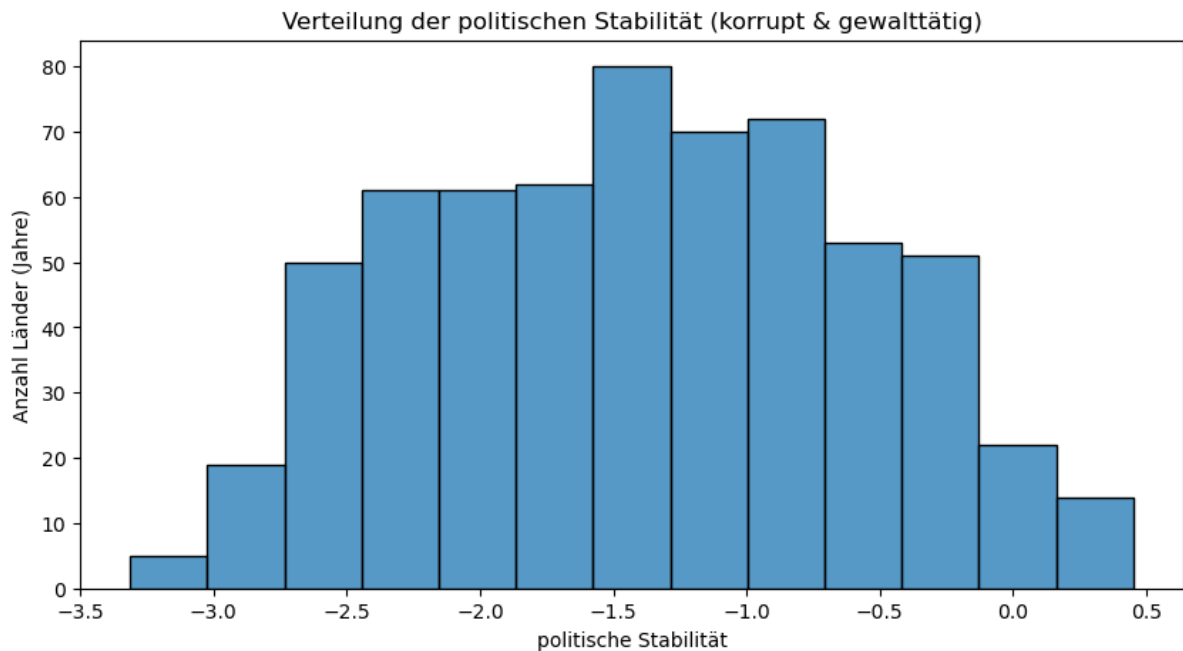
- ich möchte alle Länder mit hohen Werten in den Variablen der Korruption und in der Variable politische Gewalt etc.. in eine Gruppe überführen.
- danach möchte ich zur Veranschaulichung eine Verteilung erstellen.
- Abschließend möchte ich wissen welche Länder enthalten sind, und wie häufig.

In [308...

```
# Erstellen der Gruppe anhand ihrer quantile
gewalt_korrumperte_länder = Data[
    (Data["politische Gewalt"] > Data["politische Gewalt"].quantile(0.66)) &
    (Data["politische Korruption"] > Data["politische Korruption"].quantile(0.66))
    (Data["öffentliche Korruption"] > Data["öffentliche Korruption"].quantile(0.66))
    (Data["Korruption der Exekutive"] > Data["Korruption der Exekutive"].quantile(0
]
```

In [309...

```
#Plotten
plt.figure(figsize=(10, 5))
sns.histplot(gewalt_korrupte_länder["politische Stabilität"])
plt.title('Verteilung der politischen Stabilität (korrupt & gewalttätig)')
plt.xlabel('politische Stabilität')
plt.ylabel('Anzahl Länder (Jahre)')
plt.show()
```



- Während der Durschnitt der politischen Stabilität ohne Filterung nach politischer Gewalt und Korruptionsarten noch bei 0 lag, liegt er nun bei -1.38926. Es ist also davon auszugehen, dass die ausgewählten Länder mit hoher politischer Gewalt und ausgeprägten Korruptionsformen deutlich instabiler sind als der globale Durchschnitt, was bestätigt, dass diese Variablen starke Prädiktoren für politische Instabilität darstellen.

In [310...

```
# Wie viele Länder sind enthalten
gewalt_korrupte_länder["Land"].nunique()
```

Out[310]: 51

In [311...

```
# Welche sind enthalten
gewalt_korrupte_länder["Land"].unique()
```

```
Out[311]: array(['Burma/Myanmar', 'Russia', 'Yemen', 'Bangladesh', 'Haiti',  
                'Honduras', 'Mali', 'Pakistan', 'Sudan', 'Afghanistan', 'Kenya',  
                'Lebanon', 'Nigeria', 'Uganda', 'Venezuela', 'Cambodia',  
                'Indonesia', 'Nepal', 'Nicaragua', 'Zimbabwe', 'Guinea', 'Burundi',  
                'Central African Republic', 'Guatemala', 'Iraq', 'Liberia',  
                'Papua New Guinea', 'Sierra Leone', 'Syria', 'Türkiye', 'Ukraine',  
                'Angola', 'Armenia', 'Cameroon', 'Chad',  
                'Democratic Republic of the Congo', 'Republic of the Congo',  
                'Dominican Republic', 'Gabon', 'Georgia', 'Guinea-Bissau',  
                'Kazakhstan', 'Kyrgyzstan', 'Libya', 'Madagascar', 'Somalia',  
                'Tajikistan', 'Togo', 'Bosnia and Herzegovina', 'Comoros',  
                'Equatorial Guinea'], dtype=object)
```

```
In [312... # Wie oft ist welches Land enthalten?  
gewalt_korrupte_länder["Land"].value_counts()
```

```
Out[312]: Land
          Syria 24
          Yemen 24
          Guinea 24
          Chad 24
          Democratic Republic of the Congo 24
          Mali 24
          Cameroon 24
          Somalia 24
          Nigeria 24
          Pakistan 23
          Iraq 22
          Afghanistan 22
          Venezuela 22
          Lebanon 21
          Haiti 21
          Sudan 20
          Republic of the Congo 20
          Zimbabwe 20
          Central African Republic 19
          Bangladesh 19
          Togo 18
          Burundi 16
          Uganda 15
          Burma/Myanmar 13
          Sierra Leone 11
          Kazakhstan 8
          Türkiye 7
          Equatorial Guinea 7
          Honduras 7
          Liberia 6
          Angola 6
          Kenya 6
          Nicaragua 5
          Georgia 5
          Guatemala 5
          Libya 4
          Comoros 4
          Gabon 4
          Tajikistan 4
          Madagascar 4
          Ukraine 3
          Russia 3
          Nepal 3
          Kyrgyzstan 2
          Dominican Republic 2
          Indonesia 2
          Cambodia 1
          Armenia 1
          Guinea-Bissau 1
          Bosnia and Herzegovina 1
          Papua New Guinea 1
          Name: count, dtype: int64
```

Present 3 highlights from the exploratory data analysis in your draft.

(Commented code + bullet points, add cells below under the respective highlight headlines)

EDA Highlight 1

EDA Highlight 2

EDA Highlight 3

Wenn ich es richtig verstehe, sind die Highlights für das final.ipynb vorgesehen.

Task 5 -- Dimensionality Reduction

PCA: Conduct your analyses and transformations according to Task 5. Call the transformed dataset "data_pca". (Commented code + bullet points, add cells below)

```
In [313... import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from pca import pca
```

- Ich muss die Variable der politischen Stabilität aus dem DF für die PCA entfernen, weil sie nicht nicht Teil der erklärenden Variablen ist.
- Ferner dürfen keine NaN Werte enthalten sein.

```
In [314... Data_ohne_pS = Data.drop(columns=['politische Stabilität'])
Data_ohne_pS = Data_ohne_pS.dropna()
```

- Land und Jahr kann als index genutzt werden

```
In [315... pca_vorbereitung = Data_ohne_pS.set_index(['Land', 'Jahr'])
```

- Standardisieren der numerischen Daten

In [316...

```
# alle numerischen Spalten müssen so transformiert werden, dass sie im Mittelwert
scaler = StandardScaler()
data_pca_v = scaler.fit_transform(pca_vorbereitung)

# Es wird ein neuer DF erstellt (aus den skalierten Werten). Aber der Index bleibt
data_pca_v = pd.DataFrame(data_pca_v, index=pca_vorbereitung.index, columns=pca_vo

data_pca_v.round(2)
```

Out[316]:

		Liberaler Demokratie (Index)	Wahlen (Index)	Meinungs- und Informationsfreiheit (Index)	Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	Zugang zur Justiz	Rechtsstaatlichk
Land	Jahr						
Mexico	1996	-0.44	-0.06	0.06	-0.32	-0.93	
	1998	-0.12	0.28	0.06	-0.32	-0.93	
	2000	0.27	0.53	0.37	-0.31	-0.93	
	2002	0.32	0.62	0.45	-0.27	-0.93	
	2003	0.34	0.66	0.45	-0.27	-0.93	
...	...	...	...	...	...	...	
Hungary	2018	-0.13	-0.15	-0.31	0.82	0.71	
	2019	-0.17	-0.20	-0.33	0.81	0.71	
	2020	-0.19	-0.21	-0.45	0.75	0.71	
	2021	-0.24	-0.27	-0.51	0.77	0.71	
	2022	-0.27	-0.27	-0.55	0.78	0.65	

4013 rows × 20 columns

- Die Tabelle zeigt die standardisierten Werte der Variablen. Dieser Schritt ist wichtig, weil er sicherstellt, dass alle Variablen denselben Einfluss auf den PCA-Prozess haben. Andernfalls könnte PCA durch Variablen mit größeren Zahlen dominiert werden, was das Ergebnis verzerren würde.

In [317...

```
data_pca_v.cov().round(2)
```

Out[317]:

	Liberaler Demokratie (Index)	Wahlen (Index)	Meinungs- und Informationsfreiheit (Index)	Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	Zugang zur Justiz	Rechtsstaatlichkeit
Liberaler Demokratie (Index)	1.00	0.98	0.85	0.88	0.83	
Wahlen (Index)	0.98	1.00	0.90	0.88	0.79	
Meinungs- und Informationsfreiheit (Index)	0.85	0.90	1.00	0.84	0.72	
Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	0.88	0.88	0.84	1.00	0.92	
Zugang zur Justiz	0.83	0.79	0.72	0.92	1.00	
Rechtsstaatlichkeit (Index)	0.91	0.85	0.71	0.88	0.88	
öffentliche Korruption	-0.81	-0.73	-0.54	-0.75	-0.76	
politische Korruption	-0.79	-0.70	-0.50	-0.72	-0.76	
Korruption der Exekutive	-0.81	-0.72	-0.55	-0.74	-0.76	
Fertilitätsrate	-0.48	-0.43	-0.19	-0.44	-0.44	
Lebenserwartung	0.53	0.48	0.27	0.44	0.44	
politische Polarisierung	-0.45	-0.38	-0.29	-0.43	-0.46	
politische Gewalt	-0.49	-0.42	-0.22	-0.52	-0.54	
Versammlungsfreiheit	0.87	0.91	0.92	0.86	0.73	
Massenmobilisierung	-0.02	0.04	0.15	-0.04	-0.09	
Autokratische Bewegungen	-0.60	-0.58	-0.53	-0.61	-0.56	
Demokratische Bewegungen	-0.09	-0.01	0.18	-0.00	-0.05	
Beteiligung an staatlich organisierten Gruppen	-0.64	-0.62	-0.57	-0.55	-0.50	
Beteiligung an unabhängigen nicht-politischen Gruppen	0.46	0.45	0.49	0.47	0.47	
Freiheit von Forschung und Lehre	0.82	0.86	0.92	0.83	0.71	

- Zu sehen ist eine Kovarianzmatrix. Ein Wert von 1 bedeutet, dass die Werte identisch sind und damit gleichzeitig variieren. Also gleichzeitig steigen oder fallen.
 - Eine negative Kovarianz bedeutet, dass die eine Variable steigt, wenn die andere sinkt, und umgekehrt.
 - 0 bedeutet, dass keine lineare Beziehung zwischen den beiden Variablen besteht.
-
- Diese Infos sind wichtig für die PCA.
 - Wenn zwei Variablen eine hohe Kovarianz haben, dann bedeutet dies, dass sie ähnliche Informationen liefern. Diese beiden Variablen könnten also zu einer einzigen Hauptkomponente zusammengefasst werden.
 - Wenn zwei Variablen kaum korreliert sind (d.h., ihre Kovarianz ist nahe bei null), deutet dies darauf hin, dass sie unterschiedliche Informationen liefern. In diesem Fall kann PCA sie als getrennte Hauptkomponenten behandeln, da sie unterschiedliche Aspekte der Daten erklären.

PCA anwenden

```
In [318...  pca = PCA() # Es wird ein PCA-Objekt erstellt. Berechnet (Eigenvektoren) basierend  
pca_t= pca.fit_transform(data_pca_v) # PCA wird auf die standardisierten Daten angewendet  
pca_t = pd.DataFrame(pca_t, index=pca_vorbereitung.index) # Erstellung eines neuen DataFrames  
pca_t.round(2)
```

```
Out[318]:
```

		0	1	2	3	4	5	6	7	8	9	10	11	12	
	Land	Jahr													
	Mexico	1996	-0.84	1.26	-1.02	0.08	-0.94	-0.07	-1.11	-0.33	0.23	-0.14	1.43	-0.69	0.0
		1998	-0.30	1.25	-0.96	-0.05	-1.07	-0.04	-1.04	-0.06	-0.17	-0.06	1.52	-0.66	0.0
		2000	0.62	1.57	-1.00	-0.23	-0.69	-0.07	-1.18	0.57	-0.93	0.20	1.23	-0.46	0.0
		2002	0.91	0.92	-0.70	-0.58	-0.53	0.02	-1.17	0.15	-0.96	-0.07	0.88	-0.29	0.0
		2003	0.65	1.05	-0.77	-0.77	-0.84	-0.14	-1.07	0.14	-1.01	0.14	0.91	-0.26	0.0
	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
	Hungary	2018	-0.00	0.73	-3.00	0.09	-0.67	-1.34	0.40	-0.38	1.61	-0.99	0.42	0.12	-0.0
		2019	0.19	0.03	-2.44	-0.33	-0.22	-1.15	0.82	-0.43	1.30	-0.74	0.16	0.35	-0.0
		2020	0.28	-1.16	-1.51	-0.77	0.28	-0.96	1.51	-0.72	1.02	-0.42	-0.07	0.51	-0.0
		2021	0.06	-1.39	-1.39	-0.62	0.32	-1.00	1.33	-1.02	1.41	-0.31	-0.06	0.29	-0.0
		2022	-0.46	0.77	-3.01	-0.01	-0.70	-1.37	0.69	0.03	1.25	-0.89	0.26	0.28	-0.0

4013 rows × 20 columns

In [319...

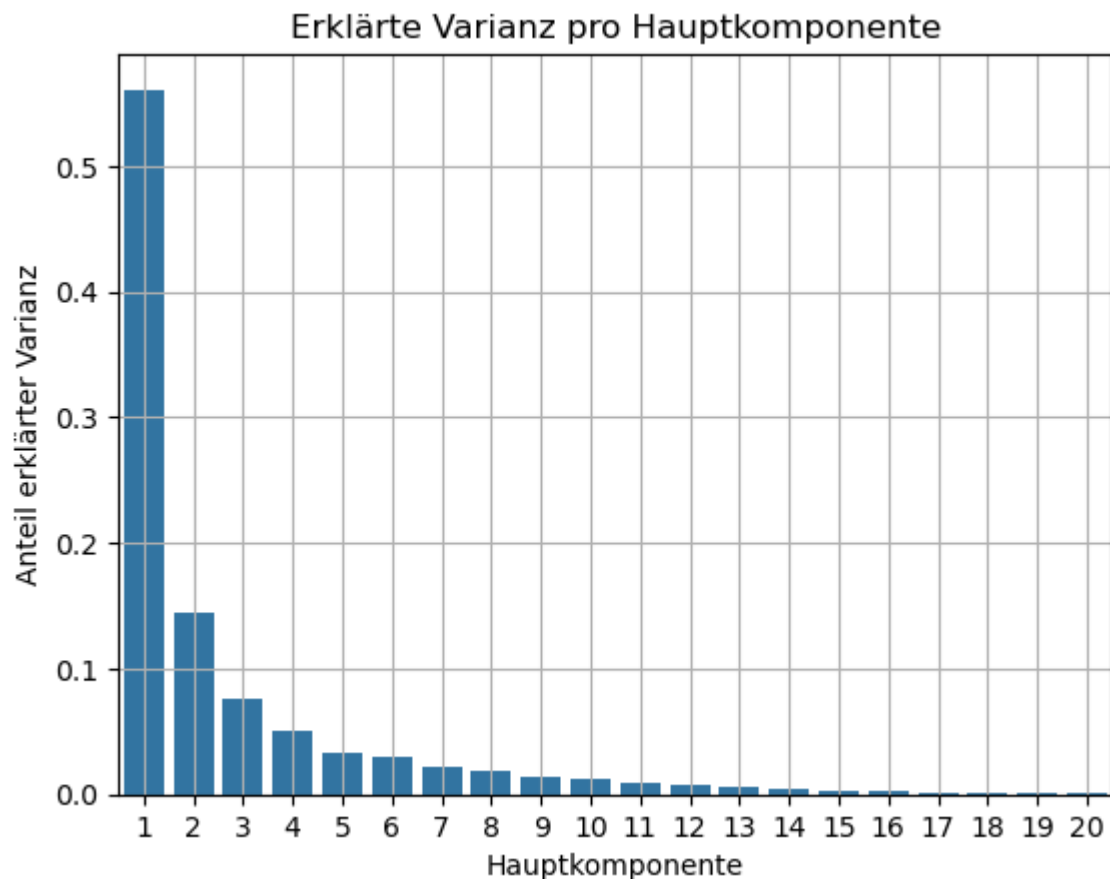
```

explained_variance = pca.explained_variance_ratio_ # --> Ich habe nur einen anderen Namen
# pca.explained_variance_ratio_ ist ein Array mit Werten zwischen 0 und 1.
# Jeder Wert steht für den Anteil der Gesamtvarianz, den eine bestimmte Hauptkomponente
sns.barplot(x=range(1, len(explained_variance)+1), y=explained_variance)
# (x=range(1, len(explained_variance)+1) --> die X-Achse wird mit 1, 2, 3, ... (statistisch)
plt.title("Erklärte Varianz pro Hauptkomponente")
plt.xlabel("Hauptkomponente")
plt.ylabel("Anteil erklärter Varianz")
plt.grid(True)
plt.show()

```

[scatterd] >INFO> Using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be plotted as numbers, cast to the appropriate data type before plotting.

[scatterd] >INFO> Using categorical units to plot a list of strings that are all parsable as floats or dates. If these strings should be plotted as numbers, cast to the appropriate data type before plotting.



- Diese Werte stellen den Anteil der erklärten Varianz pro Hauptkomponente (PC) dar. Die Summe aller Werte ergibt 1 bzw. 100% und zeigt, wie viel Information (Varianz) jede Komponente aus dem ursprünglichen Datensatz erhält.

- Typischerweise wählt man so viele Komponenten, dass 90% der Varianz erklärt sind. In meinem Fall wähle ich die ersten 7 Komponenten (91,7%).
- Die Tatsache, dass die erste Komponente (56%) so dominant ist zeigt, dass viele Variablen korrelieren. Das liegt wahrscheinlich an den Korruptionsindikatoren, die gemeinsam variieren.

Erstellen des Dataframes " data_pca mit den ersten 7 Komponenten

In [320...

```
from pca import pca
pca = pca(n_components=7)

# Fit transform
results = pca.fit_transform(data_pca_v)

#erstellen des Dfs
data_pca = results['PC']
```

```
[pca] >Extracting column labels from dataframe.
[pca] >Extracting row labels from dataframe.
[pca] >The PCA reduction is performed on the [20] columns of the input dataframe.
[pca] >Fit using PCA.
[pca] >Compute loadings and PCs.
[pca] >Compute explained variance.
[pca] >Outlier detection using Hotelling T2 test with alpha=[0.05] and n_components=[7]
[pca] >Multiple test correction applied for Hotelling T2 test: [fdr_bh]
[pca] >Outlier detection using SPE/DmodX with n_std=[3]
```

In [321...

```
data_pca
```

Out[321]:

	PC1	PC2	PC3	PC4	PC5	PC6	PC7
(Mexico, 1996)	-0.839234	1.259897	-1.023401	0.083641	-0.941285	-0.073682	-1.111952
(Mexico, 1998)	-0.295286	1.249717	-0.961652	-0.052201	-1.071970	-0.042377	-1.040639
(Mexico, 2000)	0.621686	1.570458	-0.995657	-0.231238	-0.688315	-0.067844	-1.180708
(Mexico, 2002)	0.912655	0.915655	-0.699395	-0.577630	-0.529156	0.019485	-1.171729
(Mexico, 2003)	0.651547	1.047620	-0.767735	-0.774925	-0.842099	-0.141986	-1.069083
...	...	...	...	...	...	...	...
(Hungary, 2018)	-0.002174	0.726271	-3.000299	0.089713	-0.666847	-1.343102	0.397839
(Hungary, 2019)	0.191349	0.031878	-2.442110	-0.327063	-0.221488	-1.154951	0.821701
(Hungary, 2020)	0.283939	-1.156477	-1.507022	-0.768636	0.279465	-0.962872	1.512068
(Hungary, 2021)	0.055234	-1.387803	-1.390219	-0.622130	0.318408	-0.998595	1.332672
(Hungary, 2022)	-0.463595	0.768741	-3.010604	-0.014477	-0.704605	-1.371551	0.685777

4013 rows × 7 columns

Problem: Woher weiß ich was sich hinter den PC1,PC2 etc.. verbirgt?

- Antwort: Loadings. PC1 etc.. sind Eigenvektoren. Diese bestehen aus den Loadings aller Variablen. Die Loadings wiederum sind Gewichte, mit denen jede ursprüngliche Variable zur jeweiligen Hauptkomponente (PC) beiträgt. Sie zeigen, wie stark und in welche Richtung eine Variable den Eigenvektor beeinflusst.

In [322]...

```
pca.results['loadings'].round(3)
```

Out[322]:

	Liberales Demokratie (Index)	Wahlen (Index)	Meinungs- und Informationsfreiheit (Index)	Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	Zugang zur Justiz	Rechtsstaatlichkeit(Index)
PC1	0.284	0.273	0.238	0.275	0.265	0.286
PC2	0.095	0.159	0.294	0.115	0.052	-0.016
PC3	-0.010	0.002	0.148	0.047	0.033	-0.020
PC4	-0.010	-0.089	-0.148	0.015	0.153	0.166
PC5	-0.008	-0.047	-0.069	-0.179	-0.147	0.102
PC6	-0.048	-0.035	-0.006	-0.102	-0.128	-0.183
PC7	-0.124	-0.083	-0.073	0.239	0.258	-0.015

- Positive Loadings (z. B. $+0.286$ für „Rechtsstaatlichkeit“) --> Diese Variablen sind positiv mit PC1 korreliert. Je höher diese Werte im Datensatz, desto stärker wird die Beobachtung entlang der PC1-Achse in positive Richtung verschoben.
- Negative Loadings (z. B. -0.264 für „öffentliche Korruption“) --> Diese Variablen sind negativ mit PC1 korreliert. Je höher der Wert bei diesen Variablen, desto stärker wird die Beobachtung entlang der PC1-Achse in negative Richtung verschoben.

- Ich habe mir die Loadings für alle PC- Komponenten ausgeben lassen und zusammengefasst:

PC1: Ein hoher PC1-Wert zeigt starke Institutionen, Rechtsstaatlichkeit und geringe Korruption; ein niedriger Wert das Gegenteil.

PC2: Ein hoher PC2-Wert deutet auf Staaten hin, die von aktiven demokratischen Bewegungen, einer freien Meinungsbildung und einer breiten politischen Teilnahme geprägt sind. Ein negativer Wert bedeutet das Gegenteil.

PC3: Ein niedriger PC3-Wert deutet auf Länder mit hoher Fertilität, niedriger Lebenserwartung und geringer gesellschaftlicher Mobilisierung hin, was oft mit weniger entwickelten oder politisch instabileren Staaten in Verbindung gebracht wird. Ein positiver Wert bedeutet das Gegenteil.

PC4: Ein hoher Wert in PC4 bedeutet, in diesen Ländern eine stärkere Teilnahme an staatlich organisierten Gruppen sowie zivilgesellschaftlichen Aktivitäten zu finden ist, die mit weniger Korruption einhergeht.

PC5: Ein hoher PC5-Wert steht für Länder, in denen politische Gewalt und Polarisierung besonders ausgeprägt sind. Diese Länder befinden sich vermutlich in einem Zustand tiefer politischer Fragmentierung oder sogar in einer Krise. Ein niedriger PC5-Wert zeigt Länder mit stärkerer Einbindung der Bevölkerung in organisierte Strukturen, weniger politischer Gewalt und geringerer Polarisierung.

PC6: Ein hoher PC6-Wert deutet auf Länder mit starkem Engagement in nicht-politischen zivilgesellschaftlichen Bereichen hin, begleitet von relativ hoher Lebenserwartung und möglicherweise schwächer ausgeprägter politischer Beteiligung. Es handelt sich um Gesellschaften mit sozialer Stabilität, aber ohne zwingend starke oder demokratisch legitimierte politische Institutionen.

PC7: Ein hoher PC7-Wert zeigt Staaten mit starker ideologischer Polarisierung innerhalb der Gesellschaft, oft trotz oder gerade wegen gut funktionierender Institutionen. Dies könnte auf „gespaltene Demokratien“ hinweisen – etwa reife Demokratien mit zunehmendem Populismus, Identitätspolitik oder Lagerbildung. Ein niedriger PC7-Wert hingegen kennzeichnet Länder mit stärkerer autoritärer Kontrolle, weniger gesellschaftlicher Spaltung, aber auch geringerer öffentlicher Beteiligung und eher technokratischer oder repressiver Stabilität.

Task 6 -- Clustering

Choice of Algorithm: Given the situation in the pitch and the results from the above analyses. Which algorithm (chosen from those discussed in the lectures) would YOU choose for the clustering? Explain! (Choice + 1-3 bullet points in the cell below)

My Choice: ...

- Ich würde ein soft Clustering (EM (Gaussian Mixture Model - (Vorlesung 11))) wählen. (Auch wenn wir dieses Clustering nicht in diesem Portfolio nutzen sollen) Die Beobachtungen des Vdem-Datensatzes lassen sich nicht eindeutig in ein kugelförmiges Cluster überführen und lassen sich mit bloßem Auge auch nur sehr schwer einem Cluster zuordnen. Der Hintergrund könnte sein, dass sich viele Länder in einer Transformation befinden (Demokratie --> Autokratie).
- Aus diesem Grund können Merkmale je Land stark variieren und sind volatil, weshalb ein "weiche Zuordnung" angebracht ist.
- Außerdem weisen viele Variablen auf einen Zusammenhang mit der politischen Stabilität hin. Weshalb ich eher elliptische Cluster erwarte.
- Zudem weisen die Länder wahrscheinlich Merkmale mehrerer Typen auf. Wenn ein Land politisch korrupt ist, dann ist wahrscheinlich auch die executive korrupt, sowie die Öffentlichkeit. --> GMM erlaubt Wahrscheinlichkeiten.

Requirements: In the following and for the remainder of the final phase of the exam (independent of what your choice above suggests), use * a clustering algorithm \$A\$, chosen from the family: `__k-means/k-means++__` * the evaluation metric \$E_1\$: `__Silhouette Score__` * the evaluation metric \$E_2\$: `__Davies-Bouldin Index__`.

In [323...

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
```

Comprehensive clustering analysis

Clustering: On both datasets, `data` and `data_pca`, under their respective headlines below, conduct comprehensive clustering analyses with algorithm \$A\$, using the available methodology from the module and specifically \$E_1\$ and \$E_2\$. Conduct ALL necessary steps to create the most helpful and meaningful clustering per dataset and discuss its quality. Use appropriate diagrams to demonstrate the results. (Commented code + bullet points, add cells)

Clustering Analysis of plain data

K-Means

```
In [324... # Die Spalten Land und Jahr sollten als index genutzt werden (bei allen Clustering)
# Diese sollte keinen Einfluss auf das Clustern nehmen. Das gilt für alle Clustering
Data_plain = Data_ohne_pS.set_index(['Land', 'Jahr'])
```

```
In [325... # da ich mehrere features mit unterschiedlich hohen Metriken habe, muss ich alle k
# --> hohe Werte würden sonst die Clusterbildung dominieren.
scaler = StandardScaler()
X = scaler.fit_transform(Data_plain)
# fit() berechnet Mittelwert und Standardabweichung für jede Spalte, transform() s
```

```
In [326... # Cluster mit Kmeans.

kmeans_ohne_pca = KMeans(
    n_clusters=2,           # Definition wie viele Cluster gefunden werden sollen
    random_state=42,        # Reproduzierbarkeit der Ergebnisse
    n_init=20,              # Wählt das beste Ergebnis aus 20 zufälligen Initial
    init='k-means++',       # KMeans++ zur besseren Startwertwahl, Random würde
    max_iter=300,           # Abbruch nach max. 300 Iterationen. -> Mehr als aus
    tol=0.0001              # Abbruch, wenn sich die Clusterzentren kaum noch ve
)

cluster_assignments1=kmeans_ohne_pca.fit_predict(X) # anwenden auf X, also auf die
```

```
In [327... sil_score = silhouette_score(X, cluster_assignments1) # silhouette_score
db_index = davies_bouldin_score(X, cluster_assignments1) # davies_bouldin_score
```

```
In [328... print(sil_score)
print(db_index)
```

```
0.3401012819173013
1.0875744907757732
```

```
In [329... # Wie viele Inizes befinden sich in den Clustern?
cluster_counts_Kmeans = pd.Series(cluster_assignments1).value_counts().sort_index()
cluster_counts_Kmeans
```

```
Out[329]: 0    2423
          1    1590
          Name: count, dtype: int64
```

Interpretation und Schlussfolgerungen bezüglich der Clusterqualität

- Ich habe verschieden viele Cluster (von 2 - 10) ausprobiert: Die Einstellung "2 Cluster" liefert die besten Ergebnisse. Dies gilt sowohl für den Silhouette-Scores als auch für den Davies-Bouldin Index.
- Ein Davies-Bouldin Index von 1.0876 deutet auf eine mäßige bis gute Clusterqualität hin. Je kleiner der Wert, desto geringer ist die Streuung innerhalb der Cluster.
- Der Silhouette-Score misst die Kohäsion (Wie nah sind Punkte an anderen im gleichen Cluster?) und die Separation (Wie weit sind sie von Punkten im nächsten Cluster entfernt?). Je höher der Wert, desto besser. In diesem Kontext bedeutet ein Wert von 0.3401, dass eine Clusterstruktur vorhanden ist. Diese ist jedoch nicht ideal, weshalb es überlappende bzw. unscharfe Grenzen zwischen den Clustern gibt.
- Die Tatsache, dass nur 2 Cluster die besten Ergebnisse liefern, scheint daran zu liegen, dass sich die Daten entlang eines zweidimensionalen Spektrums am besten gruppieren lassen. Es handelt sich um ein Spektrum, welches Länder vermutlich von liberal demokratisch hin zu autoritär korrupt sortiert.
- Neben der Clusteranzahl beeinflussen Parameter wie `init`, `n_init`, `max_iter` und `tol` das KMeans-Ergebnis. `init` bestimmt die Startmethode der Zentren, „k-means++“ ist meist besser als zufällig. Dies ändert hier aber nichts. `n_init` legt fest, wie oft mit verschiedenen Startpunkten neu gestartet wird, was die Ergebnisqualität verbessert. Macht hier aber auch keinen Unterschied. `max_iter` begrenzt die maximale Anzahl an Iterationen: zu klein kann zu ungenauen Clustern führen. `tol` definiert die Abbruchschwelle für die Konvergenz: ein zu hoher Wert kann die Qualität verschlechtern, ein zu niedriger verlängert die Laufzeit.
- Vorzüge Kmeans:

K-means ist gut bei „runden“ Clustern. Wenn sich Cluster wirklich als kugelförmige Gruppen im Raum verteilen, liefert K-means sehr präzise Ergebnisse. Bei länglichen Clustern, wie es hier zu erwarten ist, könnten jedoch verzerrte Ergebnisse entstehen. Der DB und der Silhouette-Score wiesen hier immerhin auf eine insgesamt mittlere bis mäßige Qualität hin. Die Cluster sind daher wahrscheinlich nicht so richtig kugelförmig.

Ferner wird jeder Punkt eindeutig einem Cluster zugewiesen, was ein Vorteil ist, da keine Rauschpunkte gibt.

Weiterhin werden die Zentren so gewählt, dass die Summe der quadrierten Abstände innerhalb jedes Clusters minimal wird. Dadurch tendieren die Cluster aber auch dazu, etwa gleich groß (in Datenpunkten) zu sein, sofern die Verteilung der Punkte halbwegs homogen ist. --> hier sind die Cluster nicht wirklich gleich groß, weshalb die Verteilung wohl auch nicht so wirklich homogen ist.

Analyse der Mittelwerte je feature + Vergleich der Zielvariable.

```
In [330... # hinzufügen der Zielvariable. Die Indexe müssen dafür gleich sein!
# Ich benötige die politische Stabilität aus Data.
Data.set_index(["Land", "Jahr"], inplace=True)

In [331... # Ich benötige die standardisierten Daten als DF.
X_kmean = pd.DataFrame(X, columns=Data_plain.columns, index=Data_plain.index)

In [332... # Diesem Df möchte ich die eine Clusterzuordnung hinzufügen.
X_kmean['Cluster'] = cluster_assignments1

In [333... # Ebenso benötigt dieser Df die Variable "politische Stabilität" aus dem original
X_kmean["politische Stabilität"] = Data["politische Stabilität"]

In [334... # Analyse der Mittelwerte je feature + Vergleich der Zielvariable.
cluster_means = X_kmean.groupby('Cluster').mean()
cluster_means
```

Out[334]:

	Liberaler Demokratie (Index)	Wahlen (Index)	Meinungs- und Informationsfreiheit (Index)	Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	Zugang zur Justiz	Rechtsstaatlichkeit(In
Cluster						
0	-0.685872	-0.650398	-0.513057	-0.593659	-0.580826	-0.67
1	1.045200	0.991141	0.781848	0.904676	0.885120	1.03

Schlussfolgerungen der Clusterergebnisse

Es lässt sich erkennen, dass der größte Unterschied zwischen den Clustern in der Variable der liberalen Demokratie, der Rechtsstaatlichkeit, der fairen und freien Wahlen und in den Korruptionsvariablen liegt.

In Cluster 0 sind Länder mit durchschnittlich niedrigen Werten bezüglich der liberalen Demokratie, der Rechtsstaatlichkeit und der fairen und freien Wahlen. Diese Länder haben folglich schlecht ausgeprägte politische Institutionen. Die Werte bezüglich ihrer Korruption sind jedoch hoch.

In Cluster 1 befinden sich Länder, die gegenteilige Ausprägungen aufweisen. Diese Länder genießen liberale Demokratie, Rechtsstaatlichkeit und freie und faire Wahlen. Gleichzeitig spielt Korruption keine Rolle.

Hinsichtlich der Zielvariable der politischen Stabilität, weisen die Cluster ebenso große Unterschiede auf. Dies bedeutet, dass das Cluster funktioniert. Cluster 1 ist politisch relativ stabil. Cluster 0 nicht. --> Die auffälligen Unterschiede zwischen den Variablen sind der Grund dafür.

DBSCAN/OPTICS

- Vorteile: Erkennt Cluster beliebiger Form

Robust gegenüber Ausreißern (werden einfach als Noise markiert)

Kein k nötig – Clusteranzahl wird automatisch gefunden

- Nachteile: Bei sehr hoher Dimensionalität (viele Features) oft schwierig einzustellen
- DBSCAN basiert maßgeblich auf zwei Parametern:

Auf dem Radius, also der Entfernung und auf der Mindestanzahl von Punkten (in diesem Radius)

Wahl der Distanzfunktion:

Ich habe DBSCAN mit Unterschiedlichen Distanzfunktionen durchgeführt.

Um die Aussagekraft zu bewerten orientierte ich mich primär an dem Silhouette-Score, Der Clusteranzahl und an der Indexverteilung.

Am Ende habe ich mich für minkowski entschieden: Die euklidische Distanz reagiert bei hoher Dimensionalität empfindlich auf Ausreißer und die Manhattan-Distanz führte bei mir zu sehr vielen Rauschpunkten (Die euklidische Distanz ebenso). Minkowski mit $p = 1.5$ bot eine ausgewogene Balance zwischen Robustheit und Trennschärfe. Hier konnte ich auch die geringste Anzahl an Rauschpunkten erzielen. Dies fand ich besonders wichtig, weil die Datenlage in den Clustern so weniger von nur ein oder zwei Ländern dominiert ist.

- Euclidean: Misst die "Luftlinie" zwischen zwei Punkten

Manhattan: Misst die Distanz entlang der Achsen (wie in einem Straßennetz)

Flexibler Abstand, der zwischen Euclidean ($p=2$) und Manhattan ($p=1$) variiert. Ich habe 1.5 genutzt. Also eine Mischung aus Euclidean und Manhattan.

```
In [335... from sklearn.cluster import DBSCAN
```

```
In [336... # Cluster mit DBSCAN --> siehe: Hilfe zum Bestimmen der Parameter:
dbscan = DBSCAN(
    eps= 3,                # Radius
    min_samples= 25,       # Mindestanzahl von Punkten im eps-Umkreis
    metric='minkowski',    # Distanzmaß (hier euklidische Distanz)
    metric_params=None,    # Zusätzliche Parameter für die Distanzfunktion (nicht
    algorithm='auto',      # Algorithmus zur Suche
    leaf_size=50,          # Blattgröße für Baum-Strukturen zur Optimierung (relev
    p= 1.5 ,               # Parameter für Minkowski-Metrik (nicht relevant bei 'e
    n_jobs=-1
)

# DBSCAN auf standardisierten Daten X ausführen
dbscan_clusters = dbscan.fit_predict(X) # X ist der standardisierter Datensatz (D

# Ausgabe Clusterlabels:
# -1 kennzeichnet Punkte, die als Rauschen (Noise) eingestuft wurden,
print(np.unique(dbscan_clusters))

[-1  0  1  2  3  4  5]
```

```
In [337... # Silhouette-Score und DB
mask = dbscan_clusters != -1 # Rauschpunkte werden nicht betrachtet.
score = silhouette_score(X[mask], dbscan_clusters[mask])
index = davies_bouldin_score(X[mask], dbscan_clusters[mask] )

print(score)
print(index)
```

```
0.13269920185787812  
0.9341676827032602
```

In [338...

```
# Wie viele Indizes sind in welchem Cluster?  
cluster_counts = pd.Series(dbscan_clusters).value_counts().sort_index()  
  
print(cluster_counts)
```

```
-1      925  
0     2866  
1       61  
2       47  
3       54  
4       35  
5       25
```

```
Name: count, dtype: int64
```

Interpretation und Schlussfolgerungen bezüglich der Clusterqualität

Um den Parameterbereich (welcher schwierig zu festzulegen war) genauer bestimmen zu können habe ich im Abschnitt "Hilfe zum Bestimmen der Parameter:" (scrollen sie etwas weiter runter) verschiedene Kombinationen ausprobiert und diese anschließend verfeinert.

Die Einstellungen eps 3 und min_samples 25 lieferten mir die besten Ergebnisse. Da es diesmal jedoch ziemlich viel Rauschpunkte gab, habe ich nicht nur versucht den silhouette_score und den davies_bouldin_score zu perfektionieren. Vielmehr habe ich versucht Die Anzahl der Indexe in den Clustern zu maximieren. Es hätte nichts gebracht, wenn ich nur viele Cluster mit ganz wenigen Indexen hätte. Diese hätten vermutlich nur zu einem Land gehört, da auf jedes Land ca. 20 Erhebungsjahre kommen.

Ferner zeigte die Clusterstruktur, dass es ein riesiges Cluster gab, welches fast alle Länder entlang der zweidimensionalen Achse erfasst. Bei diesem Cluster (0) mit 2866 Indizes habe ich den Verdacht, dass es sich um einen Ketteneffekt handelt. --> Punkte, die nicht wirklich zusammengehören, aber nacheinander „nah genug“ liegen, werden zu einer langen Kette zusammengefasst, obwohl sie nicht wirklich Ähnlichkeiten aufweisen. Neben diesem riesigen Cluster ließen sich bei dieser Einstellung jedoch auch noch einigermaßen große kleinere Cluster entdecken, die spannende Merkmale aufwiesen.

Clusterqualität

Die Tatsache, dass der Silhouette-Score schlecht ist und der Davies–Bouldin-Index recht gut, bedeutet bei DBSCAN meist, dass es wenige saubere Clusterkerne gibt. Aber auch, dass es ein großes Cluster gibt oder aber viele Rauschpunkte, die nicht wirklich passen. Deswegen hatte ich die Idee, die Rauschpunkte zu missachten. Das hat aber nicht wirklich geklappt, weil sich immer ein großes Cluster ergab, wenn ich mehr als 25 Indizes in den Clustern haben wollte. Insgesamt ließen sich jedoch 5 kleine klare Cluster erkennen.

Vergleich der Mittelwerte je Feature und Cluster

In [339...

```
X_DBSCAN = pd.DataFrame(X, columns=Data_plain.columns, index=Data_plain.index)
X_DBSCAN["Cluster"] = dbscan_clusters # ClusterLabels

# hinzufügen von politischer Stabilität
X_DBSCAN["politische Stabilität"] = Data["politische Stabilität"]

# Vergleich der Mittelwerte je Feature und Cluster
cluster_means = X_DBSCAN.groupby("Cluster").mean()
cluster_means
```

Out[339]:

	Liberales Demokratie (Index)	Wahlen (Index)	Meinungs- und Informationsfreiheit (Index)	Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	Zugang zur Justiz	Rechtsstaatlichkeit(In
Cluster						
-1	-0.703984	-0.724807	-0.718912	-0.652982	-0.529871	-0.51
0	0.321605	0.347822	0.372911	0.306802	0.247179	0.23
1	-1.170121	-1.459756	-1.741954	-1.167210	-0.992601	-0.82
2	-1.097881	-1.625272	-1.988019	-0.376888	0.254485	0.13
3	-1.261881	-1.214303	-1.618612	-1.402945	-1.501029	-1.52
4	-1.315762	-1.378707	-1.936080	-1.840102	-1.552591	-1.47
5	-1.334528	-1.886046	-1.956173	-1.848384	-1.372020	-0.79

Das beste Ergebnis mit Minkowski gelang mir bei $\epsilon = 3$ und samples 25.

- Minkowski liefert 6 Cluster und "nur" 925 Rauschpunkte.

Cluster 0: Cluster 0 ist ein großes Cluster, was sich entlang des Spektrums erstreckt. Es beinhaltet im Schnitt leicht überdurchschnittlich starke Demokratiewerte und leicht überdurchschnittliche Freiheits- und Lebenswerte, eine geringe Korruption, eine moderate Mobilisierung und insgesamt stabile Verhältnisse. --> Ist meiner Meinung nach durch den Ketteneffekt verzerrt.

Cluster 1: besteht wahrscheinlich aus Autokratien – schlechte liberale Werte, etwas Korruption, starke autokratische Bewegungen, etwas instabil.

Cluster 2: wahrscheinlich Repressive, aber gefestigte Regime – extrem niedrige Demokratiewerte, wenig Korruption, hohe Gewalt und Autokratie-Mobilisierung, paradox positiv bei politischer Stabilität.

Cluster 3: sehr niedrige Freiheits- und Justizwerte, extrem hohe Korruption, wenig offene Gewalt, politisch fragil.

Cluster 4: schwache Institutionen, sehr hohe Massen- und Protestmobilisierung (auto- und demokratisch), etwas instabil.

Cluster 5: schlechte DemokratieWerte, mäßige Korruption, kaum Mobilisierung oder Gewalt; politisch lethargisch und repressiv, recht instabil.

Die Cluster weisen interessante Eigenschaften auf. Diese sind jedoch mit Vorsicht zu genießen, da sich nicht allzu viele unterschiedliche Länder in den kleineren Clustern befinden.

Fazit:

- Die Qualität des DBSCAN- Clusters ist sehr stark von seinen Parametern abhängig. DBSCAN hängt stark von den Parametern eps und min_samples ab, weil diese festlegen, wie dicht Punkte sein müssen, um als Cluster zusammengefasst zu werden. Ein zu kleines eps und ein zu großes min_samples kann viele Punkte als Rauschen einstufen, ein zu großes eps und ein zu kleines min_samples führt zu unscharfen Clustern. Das gilt für alle Distanzfunktionen!
- Daher habe ich mir angeschaut, wie viele Indizes die Cluster enthalten und versucht das rauschen zu minimieren. Meine Ziel: möglichst wenig Rauschen, hoher Silhouette-Score, nicht zu viele Cluster. Dieses Ziel habe ich versucht mit allen Distanzfunktionen zu erreichen.
- Es war schwierig die Parameter sinnhaft einzustellen. Die Auswahl die ich getroffen habe ist wahrscheinlich nicht perfekt und von einem Ketteneffekt betroffen.
- Nichts desto trotz, ließen sich die drei Cluster hinsichtlich ihrer Mittelwerte vergleichen und lieferten in Bezug auf die Zielvariable vernünftige Gruppen von Ländern.

Hilfe zum Bestimmen der Parameter:

In [340...

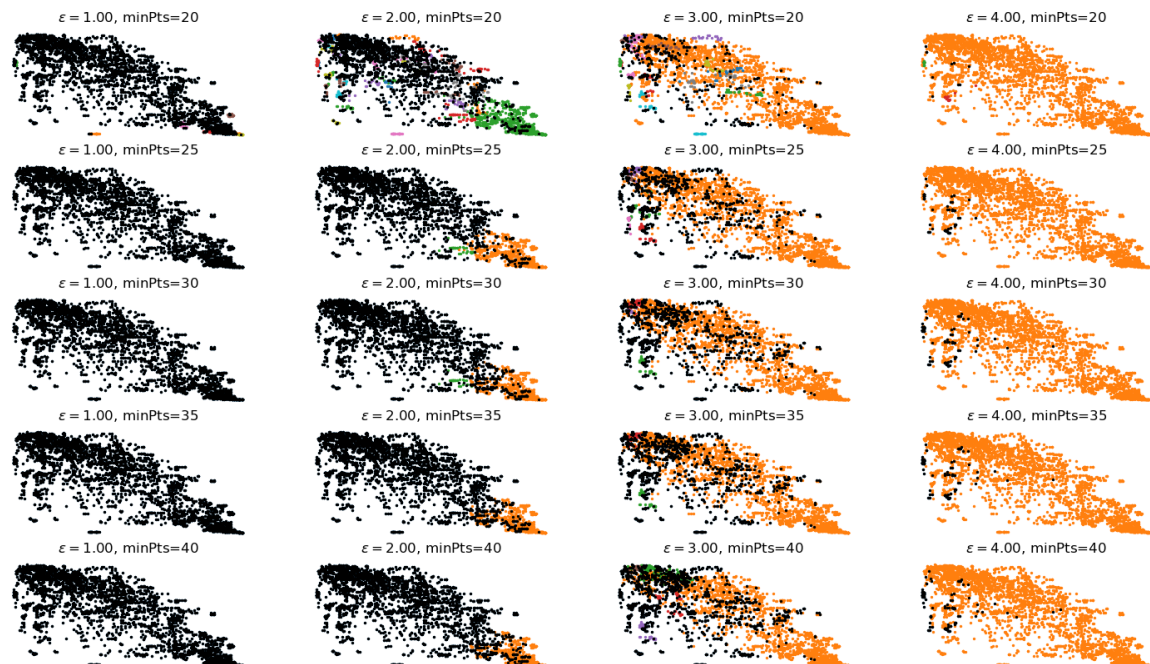
```
# Code aus Density Based Clustering

# X hat die gleiche feature Reihenfolge wie Data_plain.
feature_x = 0 # Index für das Feature auf der x-Achse = Liberale Demokratie
feature_y = 7 # Index für das Feature auf der y-Achse = politische Korruption
epsilons=(1, 2, 3, 4)
min_ptss=(20, 25, 30, 35, 40)

point_size=2

epss, ptss = np.meshgrid(epsilons, min_ptss)
_, axes = plt.subplots(nrows=len(min_ptss), ncols=len(epsilons), figsize=(18, 10))
for ax, eps, min_pts in zip(axes.reshape(-1), epss.reshape(-1), ptss.reshape(-1)):
    ax.set_axis_off()
    dbscan=DBSCAN(eps=eps, min_samples=min_pts, metric='minkowski', metric_params=
    dbscan.fit_predict(X)
    y_pred=dbscan.labels_
    for i in np.unique(y_pred):
        ax.scatter(X[y_pred==i, feature_x], X[y_pred==i, feature_y ], s=point_size)
    ax.scatter(X[y_pred==-1, feature_x], X[y_pred==-1, feature_y], c='black', s=point_size)

    ax.set_title(r'$\epsilon$={:.2f}, minPts={:i}'.format(eps, min_pts))
```



Als Hilfe um die Einflüsse der Parameter besser bestimmen zu können, habe ich diesen Code aus der Vorlesung genutzt. Das Problem: Ich kann nur 2 Features zeigen. Ich weiß also immernoch nicht, wie ich meine Parameter wählen muss. Da ich aber aus dem K means Clustering weiß, dass Es ein Kontinuum zwischen Korruption und hoher demokratischer Qualität gibt, dachte ich, dass es sinnvoll sein könnte die Features entsprechend zu wählen.

Dennoch ließ sich leider kein wirklich sinnhaftes Cluster bilden. Ein hoher Silhouette-Score ging immer mit einem hohen rauschen einher.

Wenn man sich die Cluster anschaut, lässt sich meist nur ein sehr großes Cluster erkennen. Die anderen Cluster bestehen aus sehr wenigen Zeilen bzw. Indexen.

Dennoch habe ich sehen können, welche Einstellungen mir oben Links Cluster liefern können. Also eine Gruppe mit geringer Demokratie und hoher Korruption. Das hat etwas geholfen.

HAC

Wahl der Distanzfunktion:

Da HAC Distanzen akkumulativ zusammenführt, können Ausreißer oder extrem unterschiedliche Merkmalsausprägungen den Clusterbaum (Dendrogramm) stark beeinflussen. Insbesondere bei quadratischen Distanzen wie der euklidischen. Die Manhattan-Distanz reduziert diesen Effekt und führt zu ausgewogeneren, stabileren Clustern, weil sie jedem Merkmal ein gleiches Gewicht gibt (keine Quadrierung). Aus diesem Grund und weil der Silhouettescore sowie der DB-Index bei Manhattan am besten war, habe ich mich dafür entschieden die Distanzfunktion "Manhattan" zu wählen.


```
In [341... from sklearn import cluster
from scipy.cluster.hierarchy import dendrogram
from scipy.cluster import hierarchy
```

```
In [342... # Clustern mit HAC
agg = cluster.AgglomerativeClustering(
    n_clusters = 2,          # Angabe wie viel Cluster ich am Ende erhalten möchte
    metric='manhattan',      # Distanzmaß
    linkage='average',       # Linkage-Kriterium: Distanz zweier Cluster average
    compute_distances=True   # speichert die Distanzen während des Merge-Prozess
                             # → nötig, um später ein Dendrogramm zu zeichnen
)

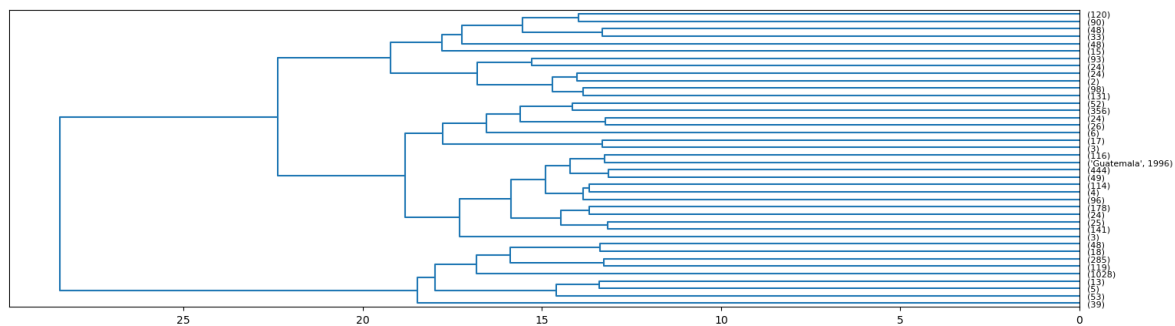
Cluster_HAC = agg.fit_predict(X) # X = Data_plain standardisiert
```

```
In [343... # Diese Zeile benötige ich um die Labels setzen zu können --> Labels=X_HAC.index
X_HAC = pd.DataFrame(X, columns=Data_plain.columns, index=Data_plain.index)
```

```
In [344... # Ich habe zu viele Zeilen für ein vollständige Dendrogramm. Daher habe ich den Code
# Der Schlüssel hierfür ist: truncate_mode='lastp', p=50. Ansonsten blieb der Code
def create_linkage(agg):
    n_samples = len(agg.labels_)
    counts = np.zeros(agg.children_.shape[0])
    for i, merge in enumerate(agg.children_):
        current_count = 0
        #print (i, merge)
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count
    linkage_matrix = np.column_stack([agg.children_, agg.distances_, counts]).astype(float)
    return linkage_matrix

matrix=create_linkage(agg)

hierarchy.set_link_color_palette(['red', 'green', 'blue'])
plt.figure(figsize=(18,5))
dendrogram(
    matrix,
    truncate_mode='lastp', p=40, # Zeigen der größten 40 Cluster.
    color_threshold=6,
    orientation='left',
    labels=X_HAC.index,
    leaf_rotation=0) # 90
plt.savefig('politischeStabilität_dendrogram.pdf')
plt.show()
```



Clusterstruktur auf Basis des Dendograms

Wie Funktioniert das Dendogram?

Jeder Datenpunkt beginnt als eigenes Cluster. Ähnliche Cluster werden zuerst fusioniert. Die Einheit für die Ähnlichkeit ist hier die Manhattan-Distanz (X - Achse). Je höher die Distanz bzw. je höher die Verzweigung stattfindet, desto unterschiedlicher sind sich die Cluster.

Die Zahlen in den Klammern geben an, wie viele Beobachtungen in diesem End-Cluster liegen. Da ich mir nur die größten 40 Cluster anzeigen lassen habe, wird mir nur die Anzahl der Beobachtungen in diesen groben Clustern angezeigt.

Auffälligkeiten:

Bei mir sind einige Cluster sehr groß (z. B. 1028), andere winzig (z. B. 2, 4); --> Unbalancierte Clusterstruktur.(Meine Daten haben eine recht ungleiche Dichte).

Die kleinen Cluster könnten Ausreißergruppen sein. Wahrscheinlich von ein und demselben Land in aufeinanderfolgenden Jahren.

Die bzw. das große Cluster könnte eine breite Masse an Ländern darstellen, die sich wenig voneinander unterscheiden.

Was sagen mir die Gabelungen?

Die höchste Gabelung liegt bei ca. 28 Manhattan Distanzen. Hier werden scheinbar zwei sehr Große Cluster zusammengeführt. Da dies die größte Distanz im Datensatz ist, scheint dies eine "natürliche Trennung" im Datensatz zu sein. Da ich bei `n_clusters = 2` auch die besten Scores (Silhouette/Davies-Bouldin) habe, ist diese Trennung scheinbar diejenige, die sich am besten und am deutlichsten begründen lässt.

P.S. wenn ich `single` einstelle ist die höchste Gabelung bei 10 Manhattan Distanzen und bezieht sich auf 2 Cluster mit unter 5 Indizes. wenn ich `complete` einstelle werden mir 60 Manhattan Distanzen angezeigt. Der DB und der Silhouette Score ist aber schlechter. Das liegt daran, dass `complete` die größten Distanzen nutzt, dadurch sind Cluster zwar weit auseinander, aber intern weniger homogen, wodurch sich der DB und der Silhouette Score verschlechtern.

In [345...

```
# Wie viele Indizes befinden sich in welchen Clustern?

# ClusterLabels hinzufügen
Data_plain_HAC = Data_plain.copy()
Data_plain_HAC["Cluster"] = Cluster_HAC

# politische Stabilität hinzufügen
Data_plain_HAC["politische Stabilität"] = Data["politische Stabilität"]
```

In [346...

```
print(Data_plain_HAC["Cluster"].value_counts()) # zählt die Zeilen
```

```
Cluster
0    2405
1    1608
Name: count, dtype: int64
```

In [347...

```
print('Silhouette Score:', silhouette_score(X, Cluster_HAC))
print('Davies-Bouldin Index:', davies_bouldin_score(X, Cluster_HAC))
```

```
Silhouette Score: 0.3297083613541679
Davies-Bouldin Index: 1.1178769549475795
```

- Bei einem Wert von $n_Cluster = 2$ beschreibt sowohl der Silhouette Score als auch der Davies-Bouldin Index die beste Clusterqualität.

0.329 bedeutet: Die Cluster sind mäßig gut trennbar und es gibt eine erkennbare Struktur. Das ist ein akzeptabler Wert für diesen Datensatz.

1.11 bedeutet: Die Cluster sind relativ kompakt, aber es gibt noch Überschneidungen oder ungleichmäßige Abstände zwischen manchen Clustern. ebenso akzeptabel für diesen Datensatz.

Linkage

Als linkage habe ich average ausgewählt.

Bei Single entsteht ein Ketteneffekt: Punkte, die nicht wirklich zusammengehören, aber nacheinander „nah genug“ liegen, werden zu einer langen Kette zusammengefasst. Das hat dazu geführt, dass ein großer Teil der Daten zu einem einzigen Cluster verschmolzen sind, obwohl intern keine echte Dichte oder Ähnlichkeit bestand. Es landeten fast alle Werte in einem Cluster.

Bei complete erhielt ich etwas schlechtere Werte bezüglich des Silhouette Score und Davies-Bouldin Index. Dies lässt sich wahrscheinlich darauf zurückzuführen, dass complete das Maximum aller Punkt-zu-Punkt-Distanzen nutzt und nicht den Mittelwert wie average. Wodurch sich der Abstand der Punkte etwas aufblähen kann.

Vergleich der Mittelwerte je Feature und Cluster

In [348...

```
X_HAC = pd.DataFrame(X, columns=Data_plain.columns, index=Data_plain.index)
X_HAC["Cluster"] = Cluster_HAC # ClusterLabels

# hinzufügen von politischer Stabilität
X_HAC["politische Stabilität"] = Data["politische Stabilität"]

# Vergleich der Mittelwerte je Feature und Cluster
mittelwerte_HAC = X_HAC.groupby("Cluster").mean()
mittelwerte_HAC
```

Out[348]:

	Libérale Demokratie (Index)	Wahlen (Index)	Meinungs- und Informationsfreiheit (Index)	Gleichheit vor dem Gesetz und individuelle Freiheiten (Index)	Zugang zur Justiz	Rechtsstaatlichkeit(In
Cluster						
0	-0.697027	-0.659952	-0.523430	-0.599844	-0.589010	-0.67
1	1.042506	0.987056	0.782866	0.897155	0.880951	1.00

Cluster 1 – Liberale Demokratien mit stabiler Ordnung Dieses Cluster zeigt stark positive Werte bei Demokratie, Rechtsstaatlichkeit, Meinungsfreiheit und Gleichheit vor dem Gesetz. Die Korruption ist deutlich negativ, was für ein hohes Maß an Integrität staatlicher Institutionen spricht. Die Lebenserwartung ist überdurchschnittlich, die Fertilitätsrate niedrig. Das ist typisch für entwickelte Länder. Politische Gewalt und Polarisierung sind gering, die Versammlungsfreiheit hoch. --> Aufgrund dieser Merkmale ist die politische Stabilität dieses Clusters hoch.

Cluster0 – Autokratische, instabilere Systeme mit hoher Korruption Die Demokratiewerte sind stark negativ, ebenso Rechtsstaatlichkeit und bürgerliche Freiheiten. Gleichzeitig ist die Korruption in allen Formen erhöht. Die Lebenserwartung ist unterdurchschnittlich, die Fertilität höher. Politische Gewalt, Polarisierung und autokratische Bewegungen sind klar positiv ausgeprägt.

→ Aufgrund dieser Merkmale ist die politische Stabilität dieses Clusters niedriger.

Fazit:

Das Dendogram liefert sehr gute Einblicke in die Clusterstruktur, wodurch sich das HAC einfacher verstehen lässt. Es ist möglich die zwei verschiedensten Cluster zu identifizieren.

Clustering Analysis of PCA transformed data

K-means

```
In [349... kmeans_pca = KMeans(
    n_clusters=2,          # Ziel: 2 Cluster finden
    random_state=42,       # Reproduzierbarkeit der Ergebnisse
    n_init=20,             # Wählt das beste Ergebnis aus 20 zufälligen Initialisierungen
    init='k-means++',      # KMeans++ zur besseren Startwertwahl
    max_iter=300,          # Abbruch nach max. 300 Iterationen
    tol=0.0001             # Abbruch, wenn sich die Clusterzentren kaum noch verändern
)

cluster_assignments2=kmeans_pca.fit_predict(data_pca) # data_pca ist schon standardisiert
```

```
In [350... sil_score = silhouette_score(data_pca, cluster_assignments2)
db_index = davies_bouldin_score(data_pca, cluster_assignments2)
```

```
In [351... print(sil_score)
print(db_index)

0.3703159581294548
1.001285823717011
```

- Ebenso wie beim Clustering mit den plain Daten liefert die Einstellung "2 Cluster" die besten Ergebnisse.
- Sowohl E1 als auch E2 haben sich jedoch leicht verbessert gegenüber dem Cluster ohne PCA. (E1: 0.37, E2:1.001)
- PCA hilft dem algorithmus also tatsächlich beim Clustern, indem es die Daten auf die relevantesten Hauptkomponenten, die den Großteil der Varianz erklären, reduziert.

Vergleich der Mittelwerte je Feature

```
In [352... data_pca_kmeans = data_pca.copy()
data_pca_kmeans["Cluster"] = cluster_assignments2 # Clusterlabels

# hinzufügen von politischer Stabilität
data_pca_kmeans["politische Stabilität"] = Data["politische Stabilität"]

# Vergleich der Mittelwerte je Feature und Cluster
mittelwerte_kmeans_pca = data_pca_kmeans.groupby("Cluster").mean()
mittelwerte_kmeans_pca
```

```
Out[352]:
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	politische Stabilität
Cluster								
0	3.515143	-0.135493	-0.101396	0.038756	0.044440	-0.078780	-0.028443	0.589006
1	-2.313894	0.089190	0.066745	-0.025512	-0.029254	0.051858	0.018723	-0.675224

Um zu verstehen, was die Mittelwerte aussagen muss man verstehen, welche Variablen in den Komponenten 1-7 dominieren. Dafür nutzt man die Loadings (siehe Aufgabe 5). Zur Erinnerung: PC1 erklärt 56% der Varianz. PC1 weißt hier den stärksten Unterschied zwischen Cluster 0 und 1 auf.

PC1: Ein hoher PC1-Wert zeigt starke Institutionen, Rechtsstaatlichkeit und geringe Korruption; ein niedriger Wert das Gegenteil.

Tatsächlich weisen auch diese beiden Cluster bezüglich der Zielvariable einen großen Unterschied auf, weshalb auch dieses Clustering, primär durch PC1 aufschlüsse bieten kann.

DBSCAN

- Vorgehen: Auch mit den PCA Daten bleibt mein Vorgehen beim Clustern mit DBSCAN gleich. Um die Aussagekraft zu bewerten orientierte ich mich primär an dem Silhouette-Score, der Clusteranzahl und an der Indexverteilung. Als Hilfe zur Parametereinstellung habe ich erneut mehrere Plots gezeichnet.

In [353...

```
dbscan = DBSCAN(
    eps= 2.5,           # Radius
    min_samples= 20,    # Mindestanzahl von Punkten im eps-Umkreis
    metric='minkowski', # Distanzmaß (hier euklidische Distanz)
    metric_params=None, # Zusätzliche Parameter für die Distanzfunktion (nicht
    algorithm='auto',   # Algorithmus zur Suche
    leaf_size=50,       # Blattgröße für Baum-Strukturen zur Optimierung (relev
    p= 1.5 ,            # Parameter für Minkowski-Metrik (nicht relevant bei 'e
    n_jobs=-1           # Nutzt alle verfügbaren CPU-Kerne für parallele Berech
)

# DBSCAN auf standardisierten Daten X ausführen
dbscan_clusters2 = dbscan.fit_predict(data_pca) # data_pca ist schon standardisie

# Ausgabe aller unterschiedlichen Clusterlabels:
# -1 kennzeichnet Punkte, die als Rauschen (Noise) eingestuft wurden,
print(np.unique(dbscan_clusters2))
```

```
[-1  0  1  2  3  4  5]
```

In [354...

```
# Silhouette-Score und DB
mask = dbscan_clusters != -1 # Rauschpunkte werden nicht betrachtet.
score1 = silhouette_score(data_pca[mask], dbscan_clusters[mask])
index1 = davies_bouldin_score(data_pca[mask], dbscan_clusters[mask])

print(score1)
print(index1)
```

```
0.13229786106199962
0.8782308724736695
```

In [355...

```
# Wie viele Indizes sind in welchem Cluster?
cluster_counts2 = pd.Series(dbscan_clusters2).value_counts().sort_index()

print(cluster_counts2)
```

```
-1      24
0     3876
1      24
2      20
3      21
4      24
5      24
Name: count, dtype: int64
```

Analyse der Cluster struktur:

Tendentiell (auf Basis von dem silhouette_score und dem DB-Index) würde ich denken, dass sich die Clusterstruktur beim DBSCAN Clustern mit PCA und die Struktur ohne PCA ähnelt. Das Problem ist jedoch, dass ich die Einstellungen nur schwierig beibehalten kann, da nicht einmal die Gleiche Clusteranzahl rauskommt. Mit der Einstellung die ich jetzt ausgewählt habe, ähneln sich die Clusterstrukturen immerhin hinsichtlich ihrer Anzahl und der Größe.

Der Silhouette-Score von 0.13 weist auf eine schwache Clustertrennung hin, da die Punkte nicht klar voneinander abgegrenzt sind. Der Davies-Bouldin-Score von 0.88 hingegen weißt auf eine mäßig bis gute Kompaktheit hin.

```
In [356... data_pca_DBSCAN = data_pca.copy()
data_pca_DBSCAN["Cluster"] = dbSCAN_clusters2 # Clusterlabels

# hinzufügen von politischer Stabilität
data_pca_DBSCAN["politische Stabilität"] = Data["politische Stabilität"]

# Vergleich der Mittelwerte je Feature und Cluster
mittelwerte_DBSCAN_pca = data_pca_DBSCAN.groupby("Cluster").mean()
mittelwerte_DBSCAN_pca
```

```
Out[356]:
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	politische Stabilität
Cluster								
-1	-4.216431	-0.308958	0.195157	0.725761	1.651806	-0.911859	-0.576248	-1.639208
0	0.108371	0.102661	0.022286	-0.037751	0.014547	0.020563	0.006431	-0.174931
1	-6.508871	-3.857180	-2.468497	0.811934	-2.246757	-1.555826	-1.425306	-0.234750
2	-1.774674	-2.250687	-1.160580	0.644357	-0.591533	-0.603077	1.790810	0.211300
3	-2.681519	-2.474960	0.613540	2.799275	0.166577	-0.844854	0.243777	-0.443429
4	-5.349661	-4.307488	-0.383997	-0.237138	-1.833383	0.254307	-1.695132	0.077958
5	2.398212	-4.064962	-0.511509	1.809842	0.426131	0.134306	0.952368	1.269417

Ich möchte auch hier nicht auf alle PC Mittelwerte je Cluster eingehen. Nur auf PC1, da dieser 56 % der Varianz der Daten erklärt und damit besonders wichtig ist. PC1: Ein hoher PC1-Wert zeigt starke Institutionen, Rechtsstaatlichkeit und geringe Korruption; ein niedriger Wert das Gegenteil.

Cluster 0: Cluster Null erstreckt sich über fast das gesamte Spektrum. Es ist kein wirkliches Cluster, sondern ein Querschnitt der Restlichen Länder.

Ansonsten unterscheiden sich die Werte bezüglich PC1 stark, was auf große Unterschiede zwischen den Clustern hindeutet. Die Tatsache, dass die Cluster klein sind macht dies jedoch wahrscheinlicher. Bezüglich der Zielvariable gilt scheinbar nur, dass eine positiver PC1 Wert mit einer politischen Stabilität einher geht. Ein niedriger PC1- Wert jedoch nicht zwangsläufig mit einer sehr geringen politischen Stabilität.

Fazit:

Auch beim DBSCAN mit PCA sind die Parametereinstellungen sehr entscheidend. Da die Cluster sich aus einem Großen und mehreren Kleinen Clustern zusammensetzen, liegt auch hier der Verdacht der Kettenbildung nahe. Ich denke, dass das Clustern mit DBSCAN einfach nicht so gut funktioniert, wenn die Daten Inhomogen sind (z.B. ein großer ichter „Haufen“ und lockere Gruppen außen).

Hilfe zur Bestimmung der Parameter:

siehe DBSCAN mit plain Data.

Diesmal habe ich mich dazu entschieden PC1 gegen PC1 zu plotten. Das ergibt natürlich eine Linie. Diese ist jedoch nicht sinnlos.

PC1: Ein hoher PC1-Wert zeigt starke Institutionen, Rechtsstaatlichkeit und geringe Korruption; ein niedriger Wert das Gegenteil.

Wenn also ein Cluster rechts zu erkennen ist, dann bedeutet das, dass es ein Cluster mit Ländern ist, die über starke Institutionen, Rechtsstaatlichkeit und eine geringe Korruption verfügen. Links müsste das Gegenteil bedeuten.

An der Länge des Clusters kann man erkennen wie viele Länder umfasst und um welche Spanne auf dem Spektrum das Cluster aufweist.

Außerdem bedeutet schwarz = Rauschpunkte. --> Ich kann also ziemlich gut sehen, welches Cluster wenig Rauschpunkte hat.

Es scheint, dass die Parameter bei eps: 2.5 am wenigsten Rauschpunkte verursachen und trotzdem noch mehrere Cluster erkennen. Wichtig war, dass diese mehr als nur 10 min_samples enthalten und damit mehr als nur 1 bis 2 Länder. Deshalb habe ich oben eps auf 2.5 und min_samples auf 20 gesetzt. In diesem Bereich ergaben diese Einstellungen die besten Werte.

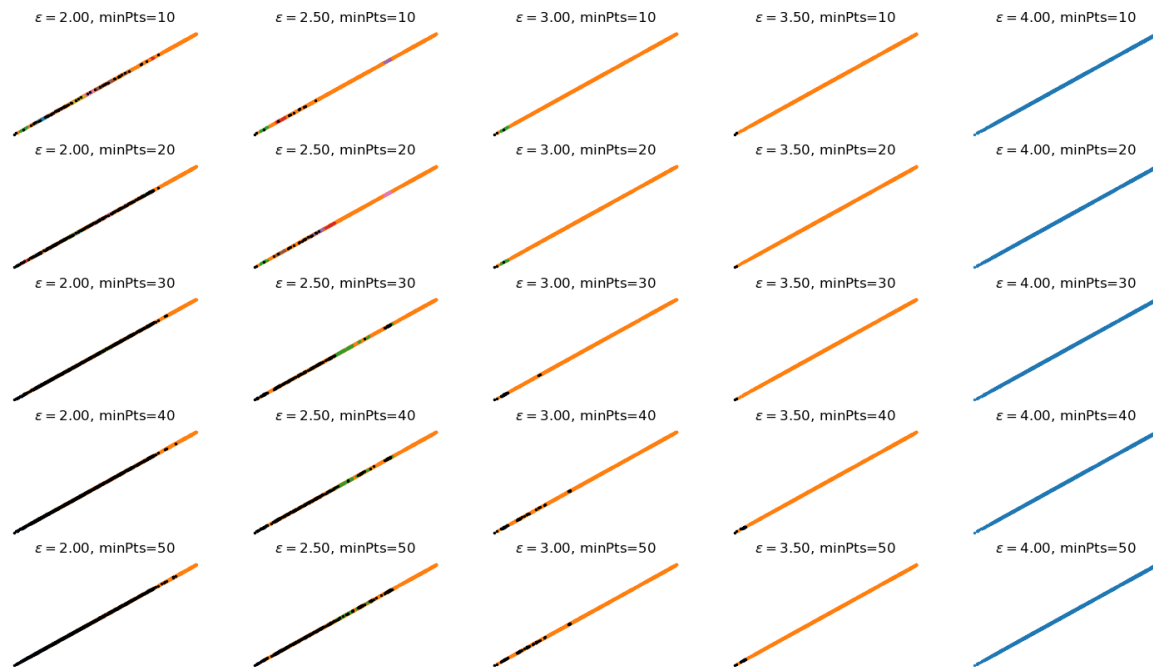
In [357...

```
# Code aus Density Based Clustering

# X hat die gleiche feature Reihenfolge wie Data_plain.
feature_x = 0 # Index für PC1
feature_y = 0 # Index für PC1
epsilons=(2, 2.5, 3, 3.5, 4)
min_ptss=(10, 20, 30, 40, 50)

point_size=2

epss, ptss = np.meshgrid(epsilons, min_ptss)
_, axes = plt.subplots(nrows=len(min_ptss), ncols=len(epsilons), figsize=(18, 10))
for ax, eps, min_pts in zip(axes.reshape(-1), epss.reshape(-1), ptss.reshape(-1)):
    ax.set_axis_off()
    dbscan=DBSCAN(eps=eps, min_samples=min_pts, metric='minkowski', metric_params=
    dbscan.fit_predict(data_pca)
    y_pred=dbscan.labels_
    for i in np.unique(y_pred):
        ax.scatter(data_pca.iloc[y_pred==i, feature_x], data_pca.iloc[y_pred==i, fe
    ax.scatter(data_pca.iloc[y_pred==1, feature_x], data_pca.iloc[y_pred==1, fea
    ax.set_title(r'$\epsilon$=%.2f, minPts=%i' % (eps, min_pts))
```



HAC

Wahl der Distanzfunktion

Die Originaldaten (ohne PCA) haben mehr Dimensionen --> das bevorzugt Manhattan.

PCA sorgt aber scheinbar dafür, dass die Daten "Runder" werden --> das bevorzugt euclidean. (Der Sil-Score und der DB-Index waren besser)

Aus diesem Grund habe ich euclidean gewählt. Zudem habe ich auch Linkage = complete gewählt, weil es „runde“ Cluster begünstigt, in denen alle Punkte eng beieinander sind.

In [358...

```
# Clustern mit HAC
agg = cluster.AgglomerativeClustering(
    n_clusters = 2, # Angabe wie viel Cluster ich am Ende erhalten möchte.
    metric='euclidean', # Distanzmaß
    linkage='complete', # Linkage-Kriterium: Distanz zweier Cluster averaged
    compute_distances=True # speichert die Distanzen während des Merge-Prozess
    # → nötig, um später ein Dendrogramm zu zeichnen
)

Cluster_HAC2 = agg.fit_predict(data_pca) # standardisiert
```

In [359...

```
print('Silhouette Score:', silhouette_score(data_pca, Cluster_HAC2))
print('Davies-Bouldin Index:', davies_bouldin_score(data_pca, Cluster_HAC2))
```

```
Silhouette Score: 0.3414153797747942
Davies-Bouldin Index: 1.0933191903640385
```

Auch die PCA-Daten beim Clustern mit dem HAC erzielen die besten Werte bei E1 und E2, wenn `n_clusters=2` gesetzt ist.

Der Silhouette-Score von 0.34 zeigt eine mäßige Clustertrennung, bei der die Cluster etwas voneinander getrennt, aber noch nicht klar abgegrenzt sind. Der Davies-Bouldin-Index von 1.09 spricht für eine durchschnittliche Clusterkompaktheit, da Werte nahe 1 typisch für überlappende oder nicht ganz klar getrennte Cluster sind.

Tatsächlich sind die Werte beim HAC-Clustering mit `pca` besser als ohne.

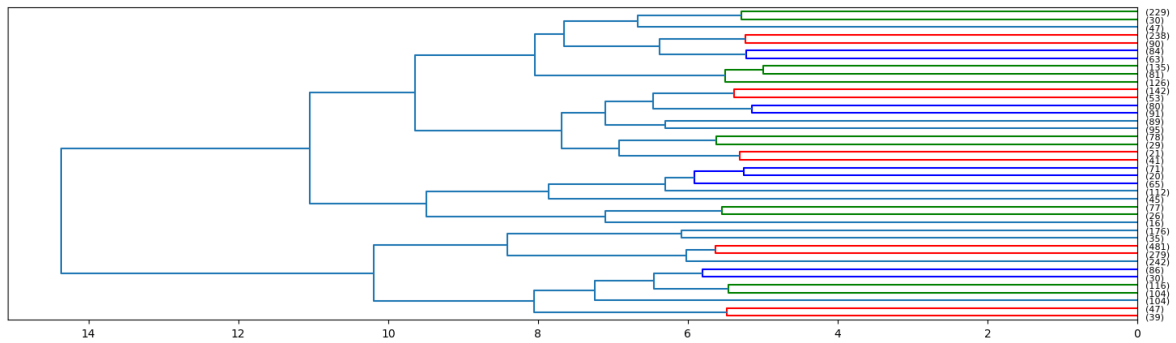
```
In [360... # Wie viele Indizes sind in welchem Cluster?
Cluster_HAC2_count = pd.Series(Cluster_HAC2).value_counts().sort_index()
Cluster_HAC2_count
```

```
Out[360]: 0    2274
          1    1739
          Name: count, dtype: int64
```

```
In [362... # Ich habe zu viele Zeilen für ein vollständige Dendrogramm. Daher habe ich den Code
# Der Schlüssel hierfür ist: truncate_mode='lastp', p=50. Ansonsten blieb der Code
def create_linkage(agg):
    n_samples = len(agg.labels_)
    counts = np.zeros(agg.children_.shape[0])
    for i, merge in enumerate(agg.children_):
        current_count = 0
        #print (i, merge)
        for child_idx in merge:
            if child_idx < n_samples:
                current_count += 1
            else:
                current_count += counts[child_idx - n_samples]
        counts[i] = current_count
    linkage_matrix = np.column_stack([agg.children_, agg.distances_, counts]).astype(float)
    return linkage_matrix

matrix=create_linkage(agg)

hierarchy.set_link_color_palette(['red', 'green', 'blue'])
plt.figure(figsize=(18,5))
dendrogram(
    matrix,
    truncate_mode='lastp', p=40, # Zeigen der größten 40 Cluster.
    color_threshold=6,
    orientation='left',
    labels=X_HAC.index,
    leaf_rotation=0) # 90
plt.savefig('politischeStabilität_dendrogram_pca.pdf')
plt.show()
```



Analyse Dendrogramm

Im Vergleich zum Clustern mit HAC ohne pca, weißt dieses Dendrogramm (mit pca) auf etwas ausgewogenere kleinste Cluster bei $p = 40$ hin, weil diesmal kein Cluster mit über 1000 Indexen heraussticht. Die Clusterstruktur ist ausgewogener.

Ferner zeigt sich, dass sich die 2 großen Cluster bei einer euclideanDistanz von 15 fusionieren. Die letzten beiden zuvor erst bei 11. Das bedeutet, dass hier zwei sehr unterschiedliche Cluster zusammengeführt werden konnten.

Die Fusionen der 3 Cluster um und bei 10 weisen auch noch auf Unterschiede hin. Auch sie könnten solide Cluster abgeben.

Nichts desto trotz, verschlechtert sich der Silhouette Score und der Davies-Bouldin Index je mehr Cluster es gibt. Die Clusterqualität scheint am besten, wenn man nur 2 Cluster auswählt.

Vergleich der Mittelwerte

```
In [363... data_pca_HAC2 = data_pca.copy()
data_pca_HAC2["Cluster"] = Cluster_HAC2
```

```
In [364... data_pca_HAC2["politische Stabilität"] = Data["politische Stabilität"]
```

```
In [365... cluster_means3 = data_pca_HAC2.groupby('Cluster').mean()
cluster_means3
```

```
Out[365]:
```

	PC1	PC2	PC3	PC4	PC5	PC6	PC7	politische Stabilität
Cluster								
0	-2.407951	0.209068	-0.006680	-0.011336	-0.061417	0.072560	0.027664	-0.720260
1	3.148753	-0.273388	0.008735	0.014824	0.080312	-0.094884	-0.036175	0.541757

Hinsichtlich der Zielvariable und auch hinsichtlich PC1 lässt sich ein starker Unterschied zwischen den Clustern erkennen.

Task 7 -- Clustering Comparison

Comparison: Compare both clustering from the previous task as described in Task 7. (Commented code + bullet points, add cells below)

Aufgabe 7 und 8 sind nur für die Finale Phase vorgesehen.

Task 8 -- Conclusions and Future Work

Subtasks: For each subtask, create a fitting headline and add additional code and markdown cells below to conduct the respective task. (Commented code + bullet points, add cells below)

In []: